



A
Project Report
on
JARVIS: AI Powered Desktop Assistant
submitted as partial fulfillment for the award of
BACHELOR OF TECHNOLOGY
DEGREE

SESSION 2025-26

in

Computer Science and Engineering

By

Aditi Agrawal (2200290100010)

Abha Gangwar (2200290100004)

Ekta (2200290100067)

Under the supervision of

Dr. Ankur Bhardwaj

KIET Group of Institutions, Ghaziabad

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

(Formerly UPTU)

May, 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Signature:

Name: Aditi Agrawal

Roll. No: 2200290100010

Date:

Signature:

Name: Abha Gangwar

Roll. No: 2200290100004

Date:

Signature:

Name: Ekta

Roll. No: 2200290100067

Date:



CERTIFICATE

This is to certify that Project Report entitled “JARVIS: AI Powered Desktop Assistant” which is submitted by Aditi Agrawal, Abha Gangwar and Ekta in partial fulfillment of the requirement for the award of degree B. Tech. in Department of Computer Science and Engineering of Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of the candidates own work carried out by them under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Dr. Ankur Bhardwaj

Associate Professor

Date:

Dr. Vineet Sharma

Dean CSE

Date:

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the B. Tech Project undertaken during B. Tech. Final Year. We owe special debt of gratitude to Dr. Ankur Bhardwaj, Department of Computer Science and Engineering, KIET Group of Institutions, Ghaziabad, for his constant support and guidance throughout the course of our work. His sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only his cognizant efforts that our endeavors have seen light of the day.

We are also thankful to the faculty members of the Computer Science and Engineering department for their support and cooperation throughout the project development process. We would like to thank our friends and colleagues for their suggestions and moral support during the completion of this project. Finally, we would like to express our heartfelt gratitude to our families for their continuous encouragement and support.

Sign:

Name: Aditi Agrawal

Roll. No: 2200290100010

Date:

Sign:

Name: Abha Gangwar

Roll. No: 2200290100004

Date:

Sign:

Name: Ekta

Roll. No: 2200290100067

Date:

ABSTRACT

Artificial Intelligence has revolutionized the interaction between humans and computers. Virtual assistants have become increasingly popular due to their ability to automate tasks and provide natural interaction using voice commands.

This project presents **JARVIS, an AI-powered desktop assistant** designed to automate everyday computing tasks through intelligent voice interaction. The assistant integrates **speech recognition, natural language processing, and automation modules** to interpret user commands and execute tasks efficiently. The system is developed using Python and includes a graphical user interface designed with **PyQt5**, which enables interactive communication between the user and the assistant.

JARVIS can perform various operations such as opening applications, retrieving information from the internet, controlling multimedia functions, and executing system commands. The architecture of the system consists of multiple layers including **input acquisition, preprocessing, intent classification, decision-making, and task execution modules**. These components work together to process user commands and generate responses in the form of text, speech, or automated actions.

Experimental results show that the assistant performs efficiently with low latency and improved responsiveness. The modular design allows future integration with advanced technologies such as large language models, IoT systems, and multilingual interface

TABLE OF CONTENTS	Page No.
TITLE PAGE.....	i
DECLARATION.....	ii
CERTIFICATE.....	iii
ACKNOWLEDGEMENT.....	iv
ABSTRACT.....	v
LIST OF FIGURES.....	ix
LIST OF TABLES.....	x
LIST OF ABBREVIATIONS.....	xi
 CHAPTER 1(INTRODUCTION).....	 1
1.1 Introduction.....	1
1.1.1 Need for Intelligent Desktop Assistants.....	3
1.1.2 Proposed System Requirement and Solution.....	3
1.1.3 Objective and Impact.....	4
1.2 Project Description.....	5
1.2.1 Functional Capabilities.....	6
1.2.2 Modular Architecture.....	7
1.2.3 Layered Architecture.....	7
1.2.4 Extensibility and Future Scalability.....	7
1.3 Objectives.....	8
1.3.1 Detailed Objectives.....	9
1.4 Scope of the Project.....	11
1.4.1 Project Scope Includes.....	11
1.4.2 Future Opportunities within the Project Scope.....	13
1.5 Project Overview.....	14
 CHAPTER 2 (LITERATURE REVIEW).....	 15
2.1 Manual Systems.....	16
2.2 Advancements in Speech Recognition and Synthesis	17

2.3 Evolution of Natural Language Processing(NLP).....	18
2.4 Role of Automation Frameworks	19
2.5 Virtual Assistant Research Perspectives.....	19
2.6 Conversational AI and Context-Aware Systems.....	20
2.7 Role of Large Language Models(LLMs).....	21
2.8 Security and Privacy Considerations	22
2.9 Performance Evaluation Metrics in Existing Research.....	22
2.10 Emerging Trends in Intelligent Systems.....	23
2.11 Extended Research Gap Analysis.....	23
2.12 Strengthening the Proposed System Position.....	24
2.13 Limitations of Existing Systems.....	24
2.14 Proposed System Contribution.....	24
2.15 Conclusion of Literature Review.....	25
 CHAPTER3 (PROPOSED METHODOLOGY).....	 26
3.1 Core Design Objectives of the Methodology	26
3.2 A Pipeline-Based Approach to Processing Data.....	27
3.3 Overview of System Architecture.....	27
3.4 Interaction Between Layers and Data Flow.....	27
3.5 Key Technical Characteristics.....	28
3.6 Input Acquisition Layer	29
3.7 Preprocessing and Normalization Layer	31
3.8 The Intent Classification and Decision-Making Layers.....	34
3.8.1 Workflow of the Processing Pipeline.....	34
3.8.2 Classification Steps.....	35
3.9 Task Execution and Automation Layer	36
3.10 System Workflow	38
3.11 Advantages of the Proposed Methodology.....	39
3.12 Methodology Limitations.....	40
3.13 Future Improvements in Methodology.....	40
3.14 Conclusion of Methodology.....	41

CHAPTER 4 (RESULTS AND DISCUSSION).....	42
4.1 Performance Evaluation Criteria.....	42
4.2 Experimental Setup.....	44
4.3 Response Time Analysis.....	44
4.4 Multitasking and Asynchronous Processing.....	45
4.5 Impact of External Dependencies.....	46
4.6 System Efficiency and Resource Utilization.....	46
4.7 Comparative Analysis.....	46
4.8 Limitations Observed.....	47
4.9 Conclusion of Results and Discussion.....	48
CHAPTER 5 (CONCLUSIONS AND FUTURE SCOPE).....	49
5.1 Conclusion.....	49
5.2 Future Scope.....	51
5.3 Final Remark.....	52
REFERENCES.....	53
APPENDIX	55
Appendix I (Sample User Commands and Interaction Flow).....	55
Appendix II (System Observations).....	56
RESEARCH PAPER.....	57
PROOF OF ACCEPTANCE.....	63
PLAGIARISM REPORT.....	64

LIST OF FIGURES

Figure No.	Description	Page No.
1.1	System Architecture and Workflow of JARVIS AI Desktop Assistant	14
2.1	Transformation of Intelligent Systems	18
2.2	Overview of Virtual Assistant Technologies and Research Gaps	25
3.1	Layered Architecture of JARVIS AI Desktop Assistant	29
3.2	Input Processing Flow in JARVIS AI Desktop Assistant	31
3.3	Query Processing and NLP Pipeline of JARVIS AI Desktop Assistant	33
3.4	Intent Classification and Decision-Making Pipeline of JARVIS AI Desktop Assistant	36
3.5	Response Generation and Output Delivery in JARVIS AI Desktop Assistant	38
3.6	Overall System Architecture and Workflow of JARVIS AI Desktop Assistant	41
4.1	Component-wise Latency Breakdown of JARVIS AI Desktop Assistant	43
4.2	Average Response Time by Query Type in JARVIS AI Desktop Assistant	45
4.3	Comparison of Asynchronous and Sequential Execution Performance in JARVIS	46
4.4	Results and Performance Analysis of JARVIS System	48

LIST OF TABLES

Table No.	Description	Page No.
3.1	Layer-wise Functional Overview of JARVIS AI Desktop Assistant	39
4.1	Impact of Concurrent Commands on JARVIS System Performance	44
4.2	Comparative Analysis of Rule-Based, Cloud-Based, and JARVIS Assistant Systems	47
4.3	Performance Metrics and Technology Mapping of JARVIS AI Desktop Assistant	47

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
GUI	Graphical User Interface
API	Application Programming Interface
LLM	Large Language Model
VAD	Voice Activity Detection
TTS	Text-to-Speech
STT	Speech-to-Text
CPU	Central Processing Unit
RAM	Random Access Memory
ML	Machine Learning
DL	Deep Learning
ASR	Automatic Speech Recognition
NER	Named Entity Recognition
HTTP	Hyper Text Transfer Protocol
IDE	Integrated Development Environment
URL	Uniform Resource Locator
DB	Database

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Artificial Intelligence (AI) and **automation technologies** have experienced rapid development, which has created new methods for users to engage with computer systems. Computing systems transitioned from simple data-processing machines to intelligent systems that can comprehend **human speech and actions and their underlying intentions** during the last three decades. The current situation developed from important advancements in **machine learning and deep learning and Natural Language Processing (NLP) and speech recognition systems**. Modern computing environments achieve higher levels of **context awareness and user responsiveness**, which improves interaction efficiency and lets users have personalized experiences.

Human-computer interaction (HCI) has typically used **Graphical User Interfaces (GUIs)** which enable users to control systems through input devices like **keyboards and mice and touchscreens**. GUIs offer effective functionality that users widely adopt yet these systems need users to perform **physical actions throughout their operation** which hampers their efficiency.

Users in environments that require **multiple tasks to be performed** or work needs to be done continuously or in urgent situations will experience lower performance levels because they need to use traditional input methods. The interfaces present **accessibility challenges** for individuals who experience **physical disabilities or temporary impairments** which restrict their ability to use the system.

There is now an increased requirement for **user-friendly interaction systems** which provide **natural and intelligent machine interfaces** that can optimize system performance while making systems easier to use. The development of **intelligent virtual assistants** represents a major advancement which provides solutions to these problems. Virtual assistants use **advanced AI technologies** that include **natural language processing and speech recognition and machine learning algorithms** to allow users to control computer systems through **natural language voice or text input**. This interaction method replicates **human communication patterns** which enables users to understand the system better than they would with standard interfaces.

Real-world applications of AI-powered interactive systems have been demonstrated through the use of current virtual assistants which include **Google Assistant and Apple Siri and Microsoft Cortana**. The assistants can execute various tasks which include **obtaining information and creating schedules and handling notifications and managing smart devices and answering questions in real time**. The integration of these devices into **smartphones and smart home systems** has improved user experience through better access to digital content which makes them vital to contemporary digital lifestyles.

Most virtual assistants that exist today depend on **cloud processing systems** to handle user requests which they send to remote servers for processing and response generation. The system provides users with access to **powerful computing capabilities and extensive data processing systems** but this system also presents multiple drawbacks. The system presents multiple challenges which include the following three problems- the system experiences **greater delays because it needs to access external networks**, and users cannot access system capabilities when they operate their devices without an internet connection. Additionally, the system creates **risks to user data protection** because it needs to transmit confidential information, and it depends on **outside services**, making it less dependable when network connections are weak.

Current virtual assistants have two major limitations which stop their systems from adapting to different user needs and providing **personalized user experiences**. The majority of

commercial systems function within **restricted operational boundaries** which stop users who want to create new features from accessing system functions. This restriction stops users from developing their **customized work processes and domain-based solutions and specialized automation functions** which professional and academic users need.

1.1.1 Need for Intelligent Desktop Assistants

The requirements for **intelligent desktop assistants** who can work in personal computing spaces need to meet the requirements for modern computing environments. The system has to provide **seamless user interaction** that lets people access both **local storage and cloud services**. The implementation of **voice interaction together with automated processes and smart decision-making systems** enables desktop assistants to decrease human work while they enhance work efficiency and boost productivity results.

The integration of **AI-powered assistants** into desktop systems corresponds with the common computing trend which creates **intelligent systems that provide users with active support throughout their work**. The advanced assistants use their understanding of the environment together with their ability to track user patterns and their ability to learn new things to create **custom user experiences**. The systems develop into **intelligent digital companions** who can predict user requirements and enhance their work execution and decision-making abilities.

The rising need for **automated solutions** that handle basic computing functions serves as a main factor which drives the creation of intelligent assistants. Users commonly execute repetitive tasks which include **starting programs, locating content, arranging documents, and controlling their computer system's components**. Intelligent assistants complete these tasks through automation which benefits users by **decreasing their time requirements and work demands while also minimizing human mistakes**.

1.1.2 Proposed System Requirement and Solution

The desktop assistant system must be **intelligent and efficient** while providing users the ability to **control tasks through voice commands**.

The system must process **various user requests** and maintain **stable performance across both internet and offline modes**.

The system uses an **expandable modular design** that enables developers to implement **future updates** such as **multilingual capabilities, advanced AI model integration, and IoT device connection**.

The project creates **JARVIS as an AI desktop assistant** that provides users with **continuous voice communication and automatic task handling** in their desktop environment. The system enables users to **control applications, handle files, obtain information, and execute commands through voice input** using a combination of **speech recognition, natural language processing, and automation systems**.

The system establishes a **modular design** which enables **future technological improvements** while supporting **system maintenance and expansion**. JARVIS combines **artificial intelligence and machine automation** with **user-centered design** to create **interactive computing systems** that help users succeed.

1.1.3 Objective and Impact

The main goal of **JARVIS** is to make **human-computer interaction simpler** which will enable users to control devices through natural methods which will make their work more efficient while using advanced intelligent systems. The system enables users to control desktop systems through **voice commands** which leads to better **productivity and accessibility and enhanced user experience**.

The project creates a **solid base** which enables future **AI-driven automation and customized console development** to progress in upcoming years. The project creates a strong base which supports future research in **AI-based automation systems and personalized computing solutions and intelligent system integration**. The project makes an important contribution to **contemporary computing** through this research.

1.2 PROJECT DESCRIPTION

The **JARVIS AI Desktop Assistant** serves as an **advanced intelligent voice-operated software tool** which enables users to operate computers through their natural speech. The system aims to create a new form of user interaction which combines **traditional desktop methods with current AI-based automation solutions** for computer usage. JARVIS provides a **user-friendly system which uses voice-based communication** instead of traditional keyboard and mouse controls which most systems need. The method enables users to interact with the system more easily while decreasing both their **mental demands and their physical needs** for standard computing activities. The system lets users operate it without using their hands which makes it useful for people who need to handle multiple tasks or who require **accessibility support or who have urgent work to complete**.

The system is developed using the **Python programming language**, which is widely recognized for its extensive ecosystem of AI, machine learning, and automation libraries. Python serves as a strong foundation for intelligent system development due to its flexibility, fast development speed, and smooth integration with external applications.

Real-time command interpretation and action execution by the JARVIS system require the integration of advanced technologies such as **speech recognition, Natural Language Processing (NLP), and automation frameworks**. The speech recognition module functions as a basic system element which takes user voice input through a microphone and transforms it into machine-readable text.

The audio signal processing system operates through multiple steps including audio signal acquisition, noise filtering, feature extraction, and speech-to-text conversion. These processes ensure accurate user command recognition, even in moderate background noise conditions.

The Natural Language Processing (NLP) module processes converted textual input and handles user command analysis by identifying syntactic structures, semantic relationships, and context-based purposes. The system uses NLP techniques such as tokenization, part-of-speech tagging, and intent classification to analyze input data and extract essential information, enabling the assistant to understand commands expressed in different ways and

improving overall user experience.

The system uses **automation frameworks together with scripting mechanisms** to perform necessary tasks after it determines user intent. The automation engine establishes direct communication with **system resources and application interfaces and web services** which enables the assistant to execute diverse tasks with high effectiveness.

The system enables users to- control applications, manage files, monitor system performance and retrieve up-to-date data.

The assistant is capable of handling both:

- **Informational queries** which require fetching weather updates and news and general knowledge
- **Functional commands** which involve starting applications and handling files and performing system tasks

JARVIS operates as an **advanced digital assistant** that provides users with both productivity tools and information access through its ability to handle both tasks.

The system uses a **Graphical User Interface (GUI)** which the **PyQt5 framework** has created to make it more user-friendly. The GUI displays how users interact with the system through their command input which generates system responses and displays process status information. The system uses **voice-based interaction together with graphical feedback** to provide users with an interactive experience which both technical and non-technical users can use.

1.2.1 Functional Capabilities

The **JARVIS assistant supports a wide range of functionalities** that assist users in their daily computing activities:

- **Application Control:** opening and closing and operating web browsers and file explorers and system utilities
- **Web Interaction:** conduct online research and obtain current information and use internet-based platforms
- **Multimedia Handling:** play audio and video content and control media playback
- **System Operations:** manage files and handle processes and monitor system

operations

- **Information Retrieval:** weather forecasts and news updates and responses to general knowledge inquiries

1.2.2 Modular Architecture

The **modular architecture of the JARVIS system** enables organizations to achieve **system flexibility and scalability and system surface maintenance**. The system operates through **independent modules** which perform specific functions while sharing data with other system parts.

The modular system design enables organizations to achieve three main benefits:

- The system allows organizations to perform **testing and repair activities**
- The system enables organizations to implement **new updates**
- The system allows organizations to introduce **new functions without causing existing system operations to stop**

1.2.3 Layered Architecture

It is a **multi-layered architecture** with the following essential components:

- **Input Acquisition Layer:** Obtain skills or information via audio and text
- **Preprocessing Layer:** Remove and Structuring: Cleans and structures the input data by removing noise and inconsistencies
- **Intent Recognition Layer:** Employs NLP methods for user command analysis and intent classification
- **Decision-Making Layer:** Assigns the order with action (if any) by based on expert rules or smart mapping
- **Execution Layer:** Executes the necessary task by automation tools, calls into APIs, system-level commands

This layered architecture allows for the **smooth and accurate flow of data**, supporting **correct processing and dependable task execution**.

1.2.4 Extensibility and Future Scalability

Besides its current functionalities, the architecture was also built with an enormous spectrum

of **extensibility and future scalability**.

WDL architectures are designed to support the integration of new technologies such as:

- Conversational Intelligence using **advanced LLMs**
- Global audience reach with **multilingual support**
- Integration of **Internet of Things (IoT)** that controls smart devices and home automation systems

Additionally, incorporating **context-aware processing, adaptive learning mechanisms, and personalized user experiences** can take the system to a whole new level of intelligent proactive assistant.

Final Overview

JARVIS AI Desktop Assistant is a **very strong versatile intelligent desktop automation tool**. The combination of **advanced AI techniques along with user-friendly interface** makes the system a great **productivity booster as well as an accessibility enhancer and improves the operational efficiency**.

This makes it:

- less dependent on manual input
- streamlines intricate operations
- provides a more **organic and interactive computing experience**

1.3 OBJECTIVES

JARVIS AI Desktop Assistant abstracts an **intelligent voice activated desktop system** by establishing a mega level **human-computer interaction (HCI)**, which allows you to execute actions through **natural language command based communication**. The whole thing is intent on reducing the dependency over **traditional input devices like keyboard and mice**, doing so will yield a more **productive, user friendly and hands free experience**.

The project also aims to provide a system that can **understand user intent, read commands and implement them intelligently in real time through AI**. It is an assistant aimed to serve as a **personal digital assistant** and can automate some repetitive operations, support the

user in **day-to-day computing tasks with automated monitoring** on its part with little input needed from the user. In addition, the system seeks to close the gap between **user intention and execution in the system itself** by allowing communication between **human language and machine-language**. And it needs to be more **responsive, reliable and adaptive** than ever for real world desktop environments.

1.3.1 Detailed Objectives

- **To Make a Real time voice interaction system**

Or design a system that can **listen to your voice through microphones**, do processing on it with as little latency as possible and give the desired response. Ensure **rapid feedback, accuracy in recognition, and a seamless dialogue flow** even with continuous use. It should also help you **handle interrupts and process commands in real-time** for greater ease of use.

- **For effective speech recognition engines to be set**

Use text technologies, including advanced of **speech-to-text technology** to convert spoken input into text. In addition complement with some audio preprocessing techniques like **noise reduction, normalization, filtering and voice activity detection (VAD)** to assist in improving recognition accuracy for environmental diversity. Another part of the goal is a consistently reliable performance regardless of accent and speech pattern.

- **To enable intent recognition via Natural Language Processing (NLP)**

Develop **NLP based modular elements** that can analyze user input using **tokenization, syntactic parsing, semantic analysis, and intent classification**; thereby providing the ability to recognize **natural language variations**, ambiguity, and different sentence **structure** when interpreting user intent accurately.

- **To enable intelligent task automation**

Design automation modules that enable the assistant to execute **system-level commands such as launching applications, managing files, executing commands, and controlling background processes**; via event-driven execution and dynamic command mapping methodology; allowing for a highly efficient and flexible way to automate tasks.

- **To enable real-time information retrieval**

Allow the system to retrieve, process, and provide **up-to-date, real-time information from**

various online sources (i.e. Search Engines, Weather Reports, News Feeds, etc.); providing **low latency response times and accurately generated responses**.

- **To develop a user-friendly GUI**

Design an easy to use, interactive GUI (using **PyQt5**) that provides users with **real-time visual feedback**, clearly shows users their commands along with their corresponding system responses and greatly improve user interaction with the system; allowing for both **voice and text input methods**.

- **For modular and scalable system architecture**

Design the system as a series of **independent, loosely coupled modules using a layered design approach**. This will allow for easier **maintenance, debugging, and scalability**, as well as enable new features and functions to be added without disrupting existing components.

- **For optimizing system performance and responsiveness**

Use **efficient algorithms and asynchronous processing** to reduce latency and increase multi-tasking capability. The system should be able to execute multiple commands simultaneously without any noticeable performance impact.

- **For ensuring reliability and robustness of the system**

Include **error handling**, exception management, and fallback response capabilities to provide consistent system performance regardless of the environment in which the system is being operated.

- **Future AI Integration Objective**

Provide a basis for future AI convergence through the creation of a system architecture that can support the integration of more modern technology, including but not limited to:

- **Large Language Models (LLMs)**
- **Multilingual processing systems**
- **Context-aware computing**
- **IoT-based automation frameworks**

The result will be an evolutionary step toward an increasingly **intelligent, adaptable assistant**.

Overall Objective Statement

Overall, the project's objective is to create an **intelligent desktop assistant** that combines **advanced AI technology with user-friendly capabilities** to create a more **interactive, intelligent, and user-centered environment**. This will allow for the advancement of **modern, cutting-edge AI-based desktop automation systems** that will enhance the user's experience compared to current systems.

1.4 SCOPE OF THE PROJECT

The **Scope of the JARVIS AI Desktop Assistant** consists of defining the **operational boundaries, functional domains, and operating capabilities** of the AI assistant for a desktop computing environment. The project will develop an **AI-based intelligent assistant** that can perform tasks via **natural language voice commands and intelligent automation** that is specifically designed for use with personal computers.

The system will operate in a **desktop ecosystem** interacting with **local resources, installed applications, operating system services, and online services**. The system will provide a **holistic solution** to automate tasks, retrieve information, and interact with users while increasing productivity, decreasing manual effort, and simplifying complex workflows.

The scope of the project encompasses **functional capabilities and technical boundaries** ensuring a well-defined environment for system implementation and evaluation.

1.4.1 Project Scope Includes

1. Natural Language Voice User Interaction

The user will have the ability to use **spoken, natural language** to provide input to the computer, eliminating the need for continuous manual input.

Benefits of this type of access to a PC are:

- Ease of use and convenience
- Operation with no hands
- Experience similar to human-human interaction

The system will provide **processed real-time voice commands** to users without delay for any user-son-user interaction, allowing for **minimal downtime due to processing delays**.

2. Speech Recognition and Natural Language Understanding

The assistant applies techniques of **natural language understanding (NLU)** that use **natural language processing (NLP)** technologies with speech recognition in converting voice inputs into commands.

Examples include:

- Speech-to-text
- Parsing of language and doing semantic analyses
- Interpreting differences in phrasing and structure of sentences

The assistant can interpret commands regardless of whether they have **different linguistic forms** or they have been stated in different manners thereby increasing flexibility and overall user experience.

3. Desktop Automation

Capabilities of desktop automation associated with JARVIS enable users to perform desktop functions such as but not limited to:

- Opening, closing and controlling multiple applications
- Navigating through various levels of the file system and controlling files and folders
- Executing multiple system-level commands (shutdown, restart, etc.)
- Monitoring performance and processes on a user's computer

4. Data Retrieval via the Internet

The assistant retrieves data from the internet to provide **real-time data access**, which may include the following (but not limited to):

- Web search results
- Weather data
- Current event data (news)
- General knowledge queries

These capabilities create an **information hub** that combines **local automation** (desktop functions) and internet-based data access.

5. Multimedia Management and Control

The assistant provides functionality to manage multimedia tasks including:

- Playing both audio and video media files
- Controlling all aspects of the playback of media (play, pause, stop, and changing

volume)

- Managing separate media applications

6. Integration with a GUI

The assistant is integrated with a **graphical user interface (GUI)** built with PyQt5.

This includes:

- A visual display of the user's commands and how the system interprets them
- Real-time status updates about the system (what it is doing)
- Allows for both text and voice interaction with the user

7. Real-time Processing and Multitasking Capability

The system was designed to have **very low latency** for executing command and to provide **real-time execution of the command**.

This is done through the use of:

- Fast processing algorithms
- Asynchronous techniques to handle multiple tasks at the same time (such as executing multiple commands at once)

1.4.2 Future Opportunities Within the Project Scope

The current solution is developed with a specific purpose, but will allow for the following further enhancements:

- Advanced AI models will be used to increase the conversational aspect of the assistant.
- Integration with IoT devices and smart systems will be accomplished to expand capabilities of the assistant.
- Multilingual Support will be added to enable the assistant to understand and communicate in multiple languages for better accessibility.
- Personalization features will be implemented so the assistant can learn user preferences and provide customized responses.
- Offline functionality will be introduced to allow basic operations without requiring an internet connection.
- Security and privacy mechanisms will be strengthened to ensure safe handling of user data and interactions.

1.5 Project Overview

In conclusion, the primary purpose of this **JARVIS AI desktop automation application** is to create **intelligent and voice automated desktop applications** using voice/voice recognition/virtual assist technology, automation of businesses, and increasing efficiencies for users creating a **seamless experience**.

Furthermore, the system is designed with a user-centric approach, focusing on simplicity, scalability, and adaptability. It integrates various technologies such as speech recognition, natural language processing, and task automation to deliver efficient and accurate performance. The application not only reduces manual effort but also enhances productivity by enabling quick execution of commands and real-time responses. Overall, the project demonstrates the potential of AI-driven desktop assistants in transforming traditional computing into a more interactive, intelligent, and efficient experience.

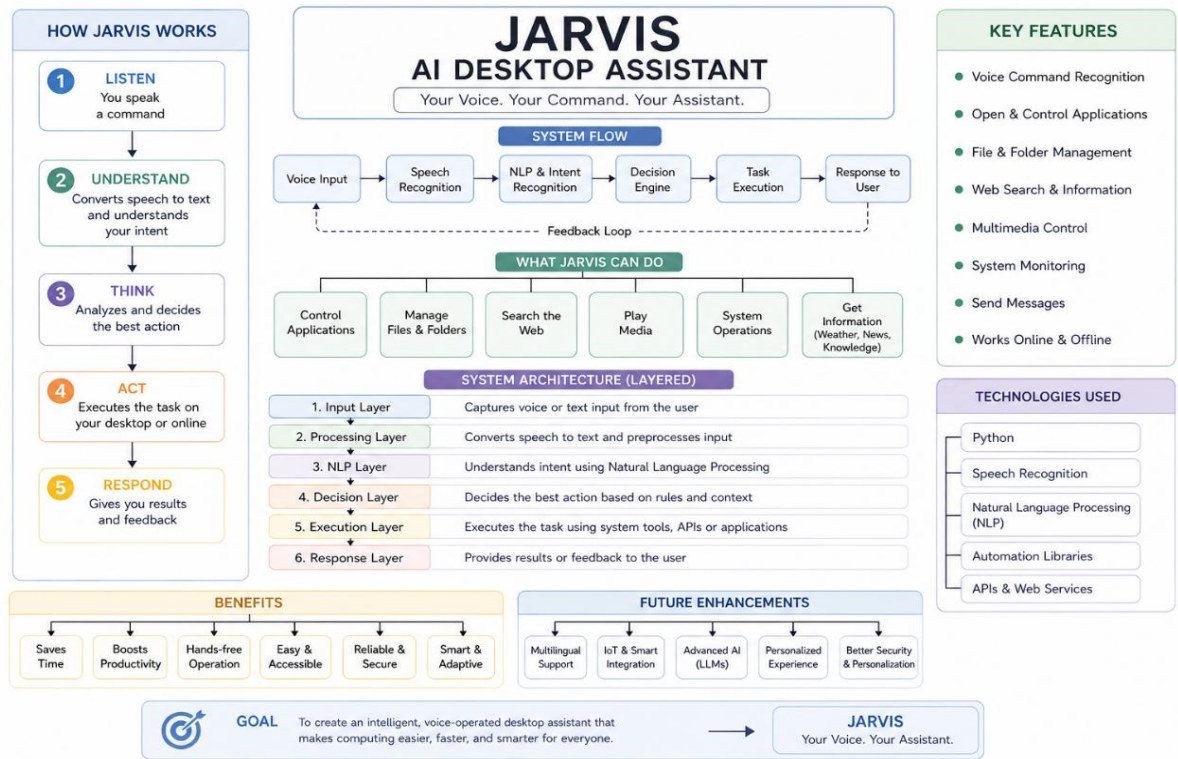


Figure 1.1: System Architecture and Workflow of JARVIS AI Desktop Assistant

CHAPTER 2

LITERATURE REVIEW

The rapid progress of Artificial Intelligence (AI), speech recognition, Natural Language Processing (NLP), and automation technologies has generated an increasing level of interest regarding the development of **intelligent virtual assistants** in recent years. The above technologies have given modern computing systems the capability to **comprehend, interpret, and reply to human language** and, as a result, significantly alter the **HCI process**. As intelligent virtual assistants integrate into more contemporary forms of communications, whether it be through **smartphones, smart home devices, or desktops**, they aid users with completing everyday tasks, increase productivity, and reduce the amount of manual effort required.

In addition to the technological developments noted above, the rise in demand for **intelligent and automatic computing systems** has driven increased levels of research in this area; users desire computing systems that not only will perform a task as directed, but will also:

- **learn user preferences**
- **modify their behaviour in accordance with those preferences**
- **provide users with proactive assistance**

The focus of research on transitioning the development of **reactive systems to proactive, intelligent assistants** in recent studies has become a primary area of interest. As a result, intelligent virtual assistants are maturing from simply being **task performers** to becoming **intelligent decision support systems** that enhance productivity and enhance users' experience.

The development of virtual assistants has taken place through **three stages - meaningful data, algorithms, and artificial intelligence**. Each of these has significantly increased the capacity of virtual assistants to **understand the needs of users**.

2.1 Manual Systems

The earliest examples of virtual assistants were **manual systems**, characterised by predetermined command sets used in conjunction with algorithmic command processing capabilities. For example, algorithms used to identify which command set should respond to an input include:

- **conditional comparison commands (if-then-else)**
- **pattern matching commands**
- **keyword recognition commands**

Although they were highly efficient and simple to develop, these systems **lacked flexibility** and could not effectively handle **complex, ambiguous, or context-rich queries**.

Joshi et al. built a voice recognition desktop assistant using Python for common tasks like opening programs and searching online and emailing someone. The Voice recognition software utilized libraries like **speech recognition** and the **text to speech (TTS) library**. Although the development of this assistant demonstrated the feasibility of using voice commands for automated processes, it was limited due to its use of **static rules-based automated processes**, which limited its ability to:

- **understand complex commands**
- **respond in different ways depending on how a user inputs the command**

Likewise, Jain et al. developed a voice controlled desktop assistant that also provided **offline speech recognition plus TTS**. This desktop assistant was different than the previous one because it also used a **graphical interface built using Tkinter** to send commands through either voice or text. Because this assistant can work without a connection to the internet, it is more reliable than the previously developed assistant in some cases. Unfortunately, the desktop assistant had a number of drawbacks:

- **limited types of automation features**
- **lacked intelligent language comprehension**

which limited its utility for performing real world tasks.

An additional limitation shared by both systems is their **lack of integration between components**, which early-stage virtual assistant research identified as a problem. In many

of the earliest implementations, modules such as **speech recognition, command interpretation, and command execution** were designed separately and were not integrated to work together efficiently. As a result, when users issued **complex or multi-step commands**, they frequently experienced **delays in receiving a response** and **inaccurate or inconsistent results**.

Furthermore, since rule-based systems are **incapable of learning**, they **cannot adapt** to new situations and **cannot improve their performance over time based on the experience gained from responding to an individual user's commands**. Due to these limitations, researchers are increasingly shifting from traditional rule-based approaches to **adaptive and intelligent systems** that utilize **machine learning techniques** to provide more flexible, accurate, and personalized responses.

2.2 Advancements in Speech Recognition and Synthesis

Improving **accuracy and performance in both speech recognition and speech synthesis** is a primary focus for researchers who are developing AI technologies. Modern speech processing systems transcend previous limitations by employing **deep learning-based models** to convert speech into text and to create **natural-sounding voice output**.

Innovative examples of such models include **Wav2Vec 2.0, Tacotron and WaveNet**. In each instance, both **vast datasets** and **sophisticated deep neural networks** are utilized to model the complex patterns in speech signals so that voice-based systems are capable of providing:

- **very high quality and accurate transcriptions**
- **highly realistic audio outputs**

While great strides have been made to date, there are still a number of challenges to improve **accuracy and sustained performance**:

- Recognition systems continue to have trouble discerning between **speech signals and background noise**, which can reduce the accurate recognition of spoken language.
- Variations in **accents, pronunciation, and patterns of speech** lead to inconsistencies in recognized vs. actual language; therefore, various models need to accommodate these distinctions.

- The **computational burden of deep learning models** continues to create challenges in terms of the ability of voice recognition and voice synthesis systems to provide **real-time performance**.

2.3 Evolution of Natural Language Processing (NLP)

The development of **Natural Language Processing (NLP)** is an important part of the role of intelligent virtual assistants and they are the technology that helps virtual assistants understand what the user has input (in terms of their command), thereby allowing **prompt action and response**. Earlier versions of NLP typically used a **keyword matching system**, where you, as the user would enter a keyword and the system would be programmed to respond to a predefined response related to that keyword.

In contrast, significant improvement in modern-day NLP has come from improvements in:

- Using **machine learning models for classifying intention**
- Using **semantic embeddings for understanding context**
- applying **deep learning architectures for building language models**

As a result of these developments, modern virtual assistant systems can **interpret complicated or ambiguously worded queries, understand and relate contextual relationships between words and phrases, and produce more relevant and accurate responses to user input**.

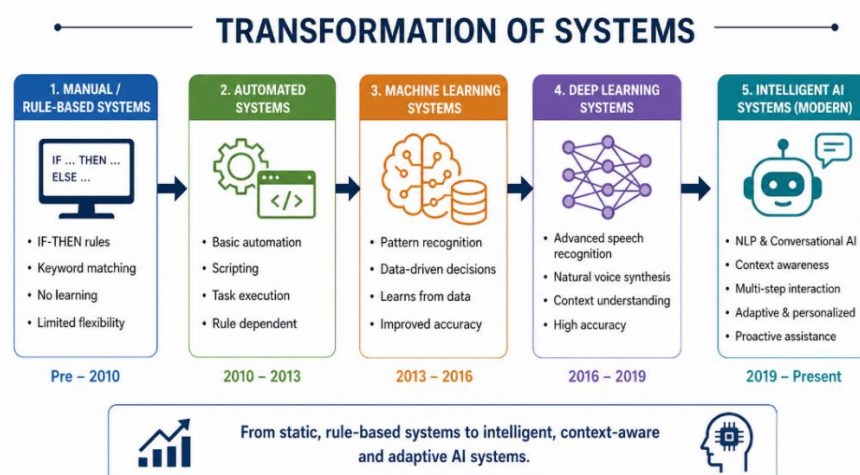


Figure 2.1: Transformation of Intelligent Systems

2.4 Role of Automation Frameworks

Automation frameworks, in addition to the use of languages to enhance the capabilities of virtual assistants, play a **significant role in improving the functionality** of virtual assistants. Virtual assistants can now directly access the **operating system, web browsers, and software programs** through the use of tools such as **Selenium, system-level scripts, and application control frameworks**. Therefore, virtual assistants can perform **different types of automation** using their automation frameworks, including automating browsers and browsing the web, performing **file management and system control**, and searching for **information in real-time**. Integrating automation into a virtual assistant's capabilities provides **added usability and practical, real-world applications** for those using virtual assistants.

Another important function of automation frameworks is the ability for virtual assistants to interact with **multiple platforms and applications** through the use of **APIs and scripting interfaces**. Virtual assistants are able to access and automate many types of applications and services through these interfaces. However, APIs and scripting interfaces present challenges such as **compatibility between systems, security permissions, and execution errors**. Therefore, when developing an effective automation solution for a virtual assistant, careful consideration must be given to **system limitations and proper exception management**.

2.5 Virtual Assistant Research Perspectives

In the last few years, **smart assistants have developed rapidly** due to the growth of **cloud-based and edge-based computing technologies**. Cloud-based smart assistants utilize **powerful remote servers** to perform processing for user commands and provide access to **large machine learning models**, enabling systems to generate results with **high accuracy and reasoning capabilities**. However, the use of cloud-based systems presents some challenges, including **network latency, concerns about data privacy, and the need for continuous internet access**.

To help solve these problems, recent studies have explored the use of **edge and hybrid computing approaches**. Using edge computing, part of the processing of user commands is done **locally on the user's device**, which results in **reduced latency, increased privacy, and improved response time** of the smart assistant's processing systems. By utilizing a

combination of **local processing for real-time tasks** and **cloud processing for complex tasks**, the hybrid computing architecture allows the smart assistant to provide a **balanced and efficient solution**. Therefore, designs of smart assistants should consider the ability to work efficiently **both when connected to the internet and when disconnected**.

Another important area of research in the field of smart assistants is the development of **distributed intelligence systems**. These systems use the **distributed nature of processing power** from local devices and cloud servers to dynamically allocate computational loads. This improves system performance by **reducing the amount of data transferred over the network, improving resource utilization, and enhancing fault recovery capability**. The design of hybrid systems requires efficient methods for **task scheduling and synchronization between local devices and cloud servers**, which is still an active area of research.

2.6 Conversational AI and Context-Aware Systems

Development of **conversational AI systems that offer more natural and human-like interactions** has advanced significantly. Traditional assistants process commands without regard for any prior interaction. However, today's advanced conversational AIs are built using **context-based approaches** and incorporate techniques like **dialogue management**, allowing the system to keep track of the **conversation's history** and provide better responses.

These approaches enable the assistant to **accommodate multiple-step queries, handle follow-up inquiries, and converse more naturally**, giving the user improved experiences. Creating such systems demands **sophisticated modeling and sufficient memory management**, which present significant challenges for current models in deploying lightweight desktop assistants.

Another significant challenge encountered within conversational AIs is preserving **long-term context and coherence** across the various interactions between the user and the assistant. Achieving context tracking on **short-term terms** can be accomplished; however, maintaining the **long-term history of a prolonged user-assistant conversational**

exchange, without reducing computational resources, is quite complex. This affects the system's ability to provide **deeply personalized and contextually aware responses** to the user over a lengthy interaction period.

2.7 Role of Large Language Models (LLMs)

Large Language Models (LLMs) have changed the intelligent assistant landscape by enabling assistants to **comprehend complex language structures, produce human sounding text responses, and perform reasoning tasks**. This new technology allows LLMs to help assistants make **intelligent decisions**, moving beyond simple command execution and enabling a more **conversational manner of interaction with users**.

Although LLMs have many **advantageous characteristics**, there are challenges that must be addressed:

- **Computational resources are required at a very high level;**
- **Large amounts of data must be gathered to train essentially each model; and**
- **There is also the need for LLMs to have a mechanism for integrating effectively into any lightweight system.**

Due to these constraints, using **Large Language Models (LLMs)** directly in lightweight or less capable systems is not possible, and therefore **optimization and modular integration strategies** must be focused on. More recently, research into optimizing LLMs for **edge deployment** has drawn significant interest. The concept is to use **small and efficient versions of large models** for edge or mobile devices.

As such, research includes techniques such as **model compression, knowledge distillation, and quantization**, which help reduce the computational requirements of LLMs while maintaining an acceptable level of performance. Each of these techniques is key to seamlessly implementing new **AI capabilities into desktop-based assistants**, such as the JARVIS assistant.

2.8 Security and Privacy Considerations

Intelligent assistants' development must now consider security and privacy as essential issues for most people. The majority of current intelligent assistants rely upon **cloud-based computing technology** that processes data and information stored remotely from the user. This reliance raises **security and privacy concerns** about the transmission of a user's data to external servers, including issues such as **confidentiality of the data being sent, access without authorization, and violations of users' privacy**.

Recent research has considered options to improve how **information about users is protected and remain functional**, including:

- **end-to-end encryption of data transferred through the intelligent assistant**
- **processing data locally on the user's device (i.e. own data not sent to external servers)**
- **providing secure authentication mechanisms (i.e. voice recognition)**

Achieving **security measures** and maintaining **system functionality** is a challenge in itself but finding the proper balance between **security and performance** is another challenge.

2.9 Performance Evaluation Metrics in Existing Research

Most researchers investigating virtual assistant effectiveness use various **performance metrics** to assess their performance. Some examples include:

- **How fast the system is at receiving and responding to requests (Response Time)**
- **How accurately the system recognizes words, and identifies what those words mean (Accuracy)**
- **How much of a delay there is in processing the request and communicating over the network(Latency)**
- **How often the system successfully completes the user's request (Task Success Rate)**
- **Overall user satisfaction with the system (User Satisfaction)**

These metrics assist researchers in **comparing different virtual assistants** and determining areas for improvement. Many virtual assistants provide **high levels of accuracy**; however, they may struggle with **latency or real-time responsiveness**, particularly when used in **low resource environments**.

In addition to the above metrics, researchers may also analyze items such as **system**

throughput (how many requests the system processes over a particular time frame), **error rate** (how often the system responds incorrectly to user input), and **scalability** (how well the system accommodates increased demand).

2.10 Emerging Trends in Intelligent Assistants

In the past couple years, researchers have noted a number of emerging trends in the design of virtual assistants. These include **personalized AI assistants**, which are designed to adjust based on individual user preferences and behaviours, and **multimodal interaction**, where systems utilize a combination of inputs such as voice, text, and visual data for greater interactivity. Another key trend is **context-aware computing**, where systems understand the user's environment and anticipate user needs accordingly.

Additionally, **IoT ecosystem integration** allows users to control smart devices through personal assistants, while **Edge AI** enables AI algorithms to run directly on devices, resulting in faster and more secure processing. These trends indicate a shift toward developing digital assistants that are not only functional but also **adaptive, intelligent, and personalized**.

2.11 Extended Research Gap Analysis

Current literature includes examples of the previous trends mentioned, and a broad overview shows a variety of progress made; however, no systems exist that have all of the below characteristics:

- **Real-time performance capability, with low latency**
- **High levels of natural language processing capability with the ability to understand context**

At the same time, systems are also expected to provide **seamless and integrated functionality within a desktop environment**, along with **adequate online and offline capabilities**, and **high customizability and modularity**. The majority of systems recorded in the published literature do not satisfy all of these requirements. Additionally, most systems do not use a combination of these criteria such as real-time response, high level of

accuracy in speech and language processing. This appears to be especially true for lightweight systems.

2.12 Strengthening the Proposed System Position

The proposed **JARVIS AI Desktop Assistant** will fill these gaps in research by addressing each of these challenges using multiple integrated approaches. These include an **efficient Speech Recognition System** that allows for real-time interaction with users, along with **Natural Language Processing (NLP)** to provide greater accuracy when interpreting user commands and understanding intents. In addition, the system incorporates **automation frameworks** that allow for the execution of practical, everyday tasks, and a **modular architecture** that promotes both scalability and flexibility of the system. Furthermore, a **hybrid processing approach** that combines both centralized and decentralized processing is used to provide greater performance, reliability, and security.

2.13 Limitations of Existing Systems

While virtual assistant systems have made great strides in recent years, they continue to have limitations. Most virtual assistants are designed to function within a **mobile or cloud-based environment** and cannot be effectively used on traditional desktop systems, resulting in poor desktop integration. Additionally, **closed architecture** limits customization for users and developers, and sensitive data being processed in the cloud raises privacy concerns. Other limitations include **lack of modular or extension capability** and **dependence on internet connection**, which restricts usability in environments with limited connectivity. These limitations highlight the need for a more **flexible, and efficient virtual assistant**.

2.14 Proposed System Contribution

This new **JARVIS desktop assistant** uses AI and integrates modern technologies to overcome the limitations of previous systems through a **modular and extensible architecture**. The system combines **speech recognition for processing voice input** and **advanced Natural Language Processing (NLP)** for understanding user intent. JARVIS also includes a **Graphical User Interface (GUI) using PyQt5**, providing both visual and

voice-based interaction.

Additionally, the system utilizes tools such as **Selenium, AppOpener, and web data extraction libraries**, enabling efficient execution of common desktop functions including **web browsing, application control, and retrieval of information from external sources**. JARVIS is designed to balance three key factors—**Intelligence, Performance, and Usability**—ensuring that advanced features do not negatively impact system response time.

2.15 Conclusion of Literature Review

There has been much progress towards creating intelligent virtual assistants but there continue to be many issues with the technologies available today, including issues with trying to provide contextual understanding, scalability, flexibility, security and desktop integration. Additionally, the ability to use the system in both offline and online environments is essential. The **JARVIS AI Desktop Assistant** meets all these criteria by providing a lightweight yet highly capable AI resource in the form of a real-time voice interactive (Voice User Interface), Intelligent Task Automation System (ITAS), and fully integrated with desktop computers.

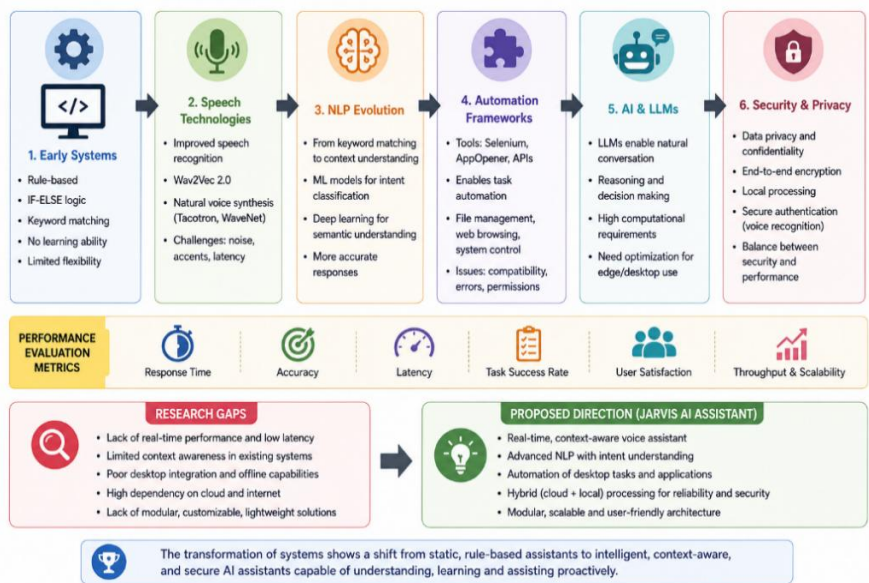


Figure 2.2: Overview of Virtual Assistant Technologies and Research Gap

CHAPTER 3

PROPOSED METHODOLOGY

The proposed system called **JARVIS: AI-Powered Desktop Assistant** uses a modular design which follows a pipeline-based architecture to convert voice and text commands from users into system actions that the system can perform. The research method establishes an intelligent interaction framework which enables users to communicate with computer systems through smooth and efficient natural language interactions.

The system operates to create a connection between human speech understanding and computer system functions, which lets users work with their systems through basic commands that boost their work speed. The design of JARVIS enables it to process natural language input while it understands what users want and carries out operations through its dynamic execution system.

3.1 Core Design Objectives of the Methodology

The main goal of this methodology seeks to create a system which meets the following criteria:

The system operates at **high speed** because it processes commands without generating any wait time. The system allows for **straightforward expansion**, which lets you add new features and technologies without any difficulties. The system operates **without any delay** when it needs to perform real-time operations. The system performs **correct user intent interpretation**, which leads to proper system action execution. The system operates **dependably** when users provide various types of input data.

The design and architecture of the system focus on:

- **Separate modules** (keeping components as independent as possible)

- **Separation of functions** (where all functions belong to an individual module)
- **Inherent structured communication between layers**

3.2 A Pipeline-Based Approach to Processing Data

- A sequential processing of data in a pipeline approach allows data to flow through various layers. Each of the layers is responsible for a specific type of transformation of the data being processed, which refines data from its original state (raw data) to its final state (meaningful and actionable instructions).
- The pipeline approach to processing data has the following advantages:
 - Easier to maintain and implement (allows for simple debugging and updating)
 - Much greater flexibility (allows modification of modules without affecting the others)
 - Better optimization/performance of each stage individually
 - Provides clarity to the data flow (allows identification of points in the pipeline where the process has been slowed))

Moreover, the pipeline uses **real time processing** so commands from users are handled with very little time lag between what they tell JARVIS to do and when it actually happens.

3.3 Overview of System Architecture

The JARVIS architecture is made up of **4 major functional layers**, responsible for executing different phases of processing user commands:

- **Input Acquisition Layer**
- **Pre-Processing and Normalization Layer**
- **Intent Classification and Decision Making Layer**
- **Task Execution and Automated Layers**

Each layer functions independently but is structured to communicate with adjacent layers using defined data interfaces to ensure **synchronized, smooth processing**.

3.4 Interaction Between Layers and Data Flow

The flow of interaction between the different layers is done in a **top down manner** (e.g.,

starting with the Input Layer receiving a user command through to the Execution Layer completing the requested task) and includes:

- The Input Layer receiving a command from end user
- The Pre-Processing Layer cleaning and structuring the input
- The Intent Layer determining the meaning of the input as well as deciding what action to take actions
- The Task Execution/Automation Layer completing the task requested

Once the task is completed, the results are sent back to the end user via:

- Voice response using Text to Speech software
- Graphical representation of the task results using a web browser

The **bi-directional nature of this interaction** not only ensures that the commands issued by the end user are completed, but also allows the end user receive feedback on the execution of their command and improves overall usability and transparency of the system.

3.5 Key Technical Characteristics

1. Modular Architectures: Each level of the architecture has been created as an Independent Unit, meaning they can be **updated easily** and **maintained independently** of all other levels of the Enterprise Architecture. Some components may be **replaced without affecting the entire system**, and new technologies can be added easily (such as advanced NLP models).

2. Scalable Design: The architecture has been designed to support **adding new** Commands and Functionality, integrating with External APIs and Services, and scalability to IOT Devices or Cloud Based Systems.

3. Real-Time Processing Capability: The architecture has been optimized for **real-time interaction** in this way: Commands will be executed with the **least amount of delay possible**, the architecture will **respond quickly to the User's commands**, and the User will experience a smooth operating environment as they interact with the application.

4. Fault Tolerance and Error Handling: The architecture has mechanisms for the **detection of non-valid Commands** (or incomplete Command), **requesting Clarification of the Command from the User**, and **gracefully handling failures in the execution of the Command**. This will enhance the **reliability and robustness** of the overall application and architecture.

5. Support for Multiple Modes of Interaction: The architecture currently supports **Voice as the primary method** of inputting a Command and **Text as a fall-back method** of inputting a Command via GUI. Therefore, the User will be able to interact with the Application in a **variety of settings and conditions**.

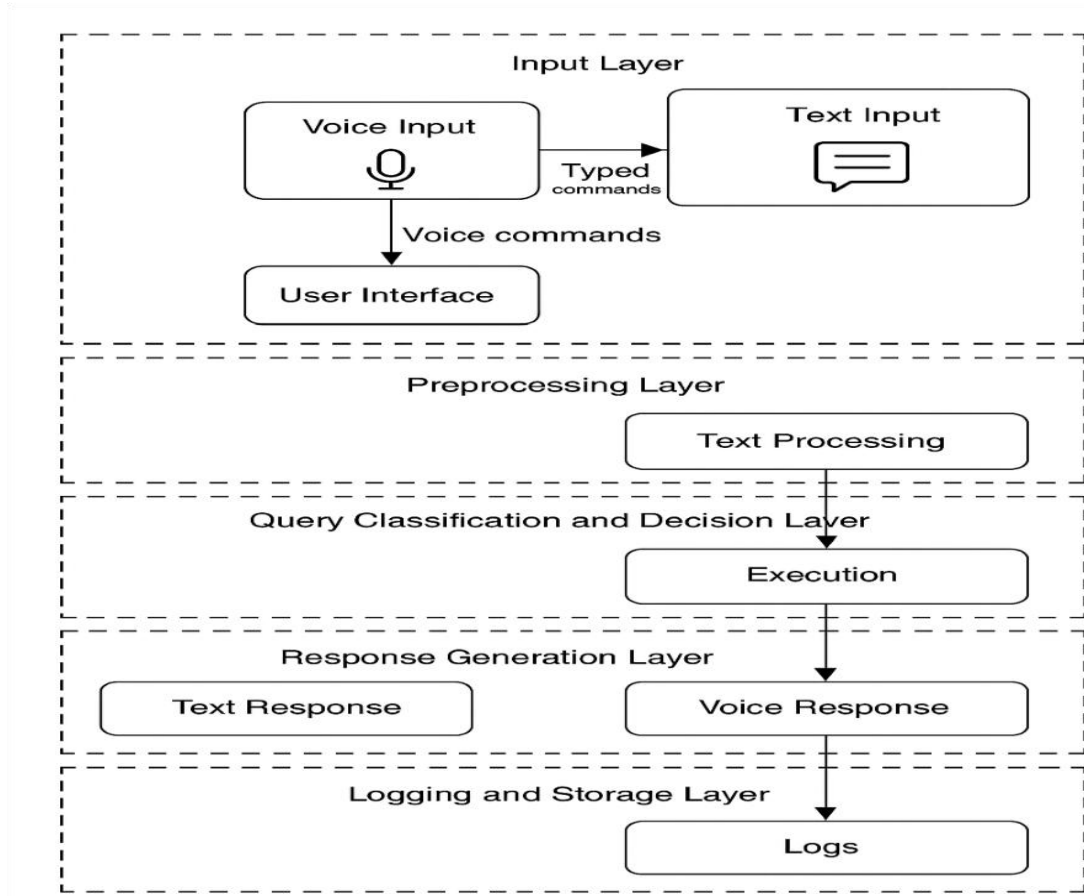


Figure 3.1: Layered Architecture of JARVIS AI Desktop Assistant

3.6 Input Acquisition Layer

The **Input Acquisition Layer** of the JARVIS System serves as the **interface for users to provide input** via either speech or typing, which will then be converted into a format that

can be **processed by machines**. This layer plays an **essential role** in ensuring the system receives **high-quality, reliable, and accurate data**, directly affecting how well it operates in later stages of processing.

- **Multi-modal Input Support:** The Input Acquisition Layer allows users to communicate with JARVIS using **multiple input types**, including **audio (voice-based)** and **text (typed)**. This provides **flexibility, accessibility, and usability** across different environments, allowing users to interact based on their unique needs.
- **Voice Input Processing:** The input received via voice is the **primary mode of communication** with the JARVIS System. Voice input goes through several stages to ensure it is **accurately captured and understood**. Audio signals are captured through the **microphone hardware interface**, which continuously monitors user activity. After capture, the analog signal undergoes **signal digitization**, converting it into a digital format (typically **16 kHz or higher**) so it can be processed further. Audio data is processed in **real time**, meaning it is handled as it is received, ensuring **minimal delay** between user input and system response. Processing is often done in **frames**, making the data manageable and enabling smooth interaction.
- **Input Quality Enhancement Techniques:** To ensure **accuracy and robustness**, several preprocessing techniques are applied. These include **Voice Activity Detection (VAD)** to detect speech and remove silence, **noise suppression algorithms** to reduce environmental noise, and **audio buffering with frame management** to ensure continuous processing. The system also uses **dynamic threshold adjustment** to adapt to noise levels and may include **echo cancellation** to reduce feedback during simultaneous input and output. Together, these techniques improve the **signal-to-noise ratio**, resulting in more accurate speech recognition.
- **Text Input Processing:** Text input serves as a **reliable alternative** to voice commands. A **Graphical User Interface (GUI)** built using **PyQt5** provides a **chat-like interface** where users can type commands and receive responses in real time. This mode is particularly useful in **noisy environments**, during **microphone failures**, or when users

prefer typing over speaking.

- **Input Validation and Standardization:** Before forwarding input to the next layer, the system ensures data is **validated and standardized**. This includes **verifying inputs** to ensure they are not empty or incorrect, **standardizing formats** into consistent text, and **detecting errors** such as incomplete or vague commands. This ensures that only **clean, structured, and valid data** is passed forward.
- **Layer Output:** Once processing is complete, all inputs are converted into **clean, machine-readable text**. This processed text is then forwarded to the **Preprocessing and Normalization Layer** for further analysis and execution.

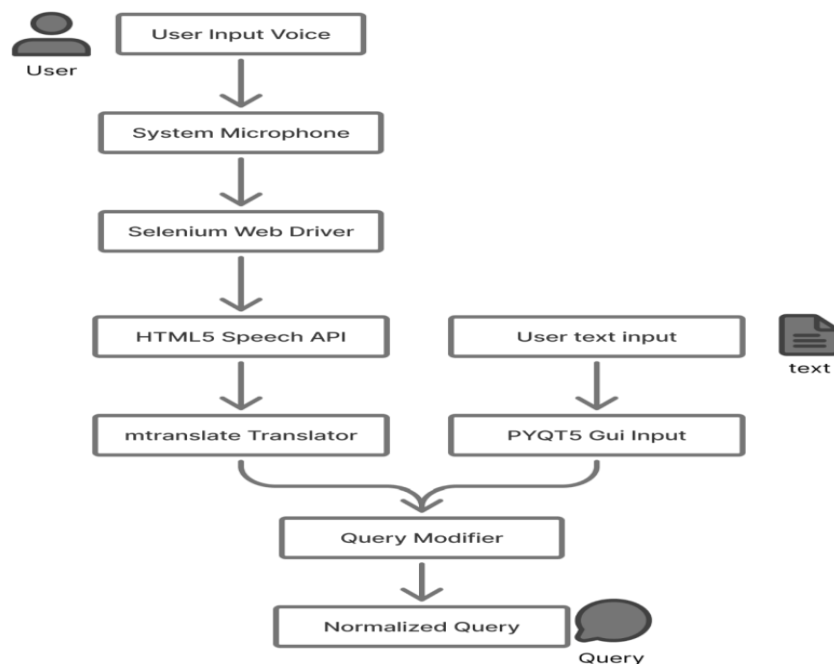


Figure 3.2: Input Processing Flow in JARVIS AI Desktop Assistant

3.7 Preprocessing and Normalization Layer

This paragraph outlines the purpose of the **Preprocessing and Normalizing layer** as taking the **raw input from the Input Acquisition layer** and transforming it into a **structured format** that will allow for further analysis. By **reducing noise** and providing a more

uniform input for further processing, this layer increases **accuracy, efficiency and consistency**.

The Preprocessing layer is particularly important when the user inputs information in a **voice to text speech recognition format**, as the input can contain **irregular and erroneous expressions as well as a variety of expressions**; therefore Preprocessing serves to make the data **uniform and intelligible** before being passed to the **Intent Classification module**.

Core Processing Steps

1. Text Normalization:

Text Normalization will convert the input data into a **uniform format** such that all commands will be processed consistently. The normalization process includes **changing all text input to lowercase, removing all punctuation and special characters, correcting for excess spaces and formatting inconsistencies, and changing all contractions to their expanded form** e.g. don't to do not. Text Normalization creates a uniform input format and permits reliable subsequent processing.

2. Tokenization:

Tokenization refers to the process of **separating the input text into smaller pieces called tokens** e.g. words or phrases. For example, the phrase **"open my browser"** will be separated into three tokens: **[open, my, browser]**, therefore simplifying Analysis and allowing the System to process each Token individually.

3. Stop Word Removal:

This removes all **common or mundane words** from the command that do not add any value to the meaning of the command. Examples of common words removed include **"the," "is," "please," and "can you"**, meaning less processing time is needed for unnecessary words and allowing focus on the keywords provided.

4. Noise and Artifacts Removal:

This removes anything unwanted from a command that may result from a **failure in speech recognition**, such as **filler words (e.g., "uh" or "um")**, **repeated words**, and **unrecognized characters/symbols**, ensuring only **relevant and meaningful information** is present after filtering.

5. Stemming and Lemmatization:

This reduces a word's size to the **base/root word form** to improve its meaning when interpreted. Examples include **"running" to "run"** and **"opened" to "open"**, allowing for better matching of commands regardless of variations in spelling.

6. Language Detection and Translation:

The system will determine the **language of the command** and convert it to a **standard language (English)** using a translation API. This allows **multilingual capability** and increases the number of potential users, including those who speak multiple languages.

- **Data Flowing to the Next Layer**

After the data is **refined and structured**, it is sent to the **Intent Classification and Decision Making Layer**, where the system examines what commands were given and makes an appropriate decision for the command.

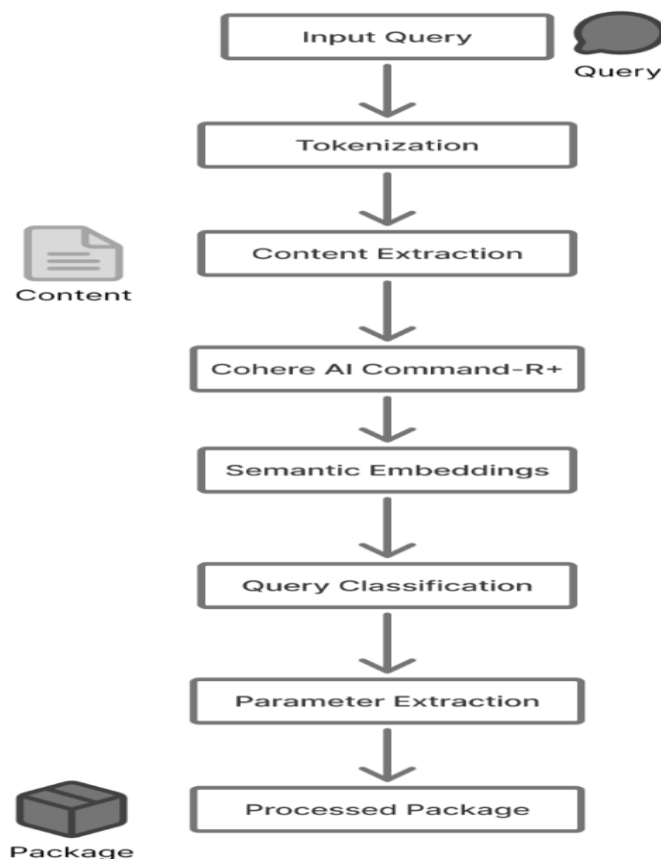


Figure 3.3: Query Processing and NLP Pipeline of JARVIS AI Desktop Assistant

3.8 The Intent Classification and Decision-Making Layers

The **Intent Classification and Decision-Making Layers** represent the **fundamental intelligence of the JARVIS system**; they are responsible for interpreting the **semantic meaning and the user intent** behind the processed inputs and how they will be used within the context of the JARVIS system. The Intent Classification and Decision-Making Layers are a **bridge from natural language understanding to actionable system commands**; without these layers, requests from users would not be correctly identified or executed by the JARVIS system.

The Intent Classification and Decision-Making Layer is responsible for **process transforming structured textual inputs into logical decisions**; thus, this layer provides the conclusions about **what a user's request is and how the assistant should respond** to that request. The efficacy of the Intent Classification and Decision-Making Layers significantly affects the **overall functionality and usability** of the JARVIS system.

3.8.1 Workflow of the Processing Pipeline

The processing pipeline within the Intent Classification and Decision-Making Layers consists of several **analytic stages**.

1. Keyword and Entity Extraction:

During this stage, JARVIS extracts the **key components** from the processed input, including **action verbs, object references, and named entities**. Action verbs describe what the system will perform (e.g., open, play, search, close), object references indicate the target such as browser, file, or application, and named entities include specific identifiable items like applications, files, or websites. This stage represents the **transformation of raw text into structured semantic elements** that are essential for identifying the user's intent.

2. Discovering an Intention:

Using a mixture of techniques, the intent is determined based on the input. These include **rule based pattern matching, predefined command mapping, and machine learning based classification**. Rule based methods provide faster results, while machine learning enables the system to respond to more unique and varied inputs. Together, these approaches allow both **speed and adaptability** in understanding user intent.

3. Evaluating Context:

Context evaluation provides additional understanding of what the user is trying to communicate by analyzing the **structure of the phrase**, the **relationship between words**, and any **previous commands** if available. This is especially useful for ambiguous requests, multi-step commands, and conversational inputs. For example, in the command “Open it”, the system uses previous context to determine what “it” refers to.

3.8.2 Classification Steps

The system classifies each command into categories so they can be processed efficiently. These categories include **queries for information retrieval**, **application control commands**, **system operations commands**, **multimedia commands**, and **web automation tasks**. This categorization enables **faster and more accurate decision-making**.

- **Decision Engine:**

After identifying the user intent, the decision engine determines the appropriate action using a **hybrid decision approach**, which combines rules and intelligent mapping techniques to ensure quick and reliable decisions. It also resolves ambiguities by requesting clarification from the user or by offering choices.

- **Confidence Evaluation:**

The decision engine assigns a **confidence score** to the recognized intent. If the confidence is high, the command is executed directly; if medium, confirmation is requested; and if low, clarification is required.

- **Advanced Capabilities:**

The system includes several enhancements such as **context-aware processing**, which remembers past interactions, **machine learning-based intent predictions**, which improve over time, a **dialogue management system** for extended conversations, and **adaptive learning** to understand user preferences.

- **Data Output of the Layer:**

When processing is completed, the system generates a **structured command object** containing the operation type, the activity to be completed, and the required parameters. This output is then forwarded to the **task execution layer** for execution.

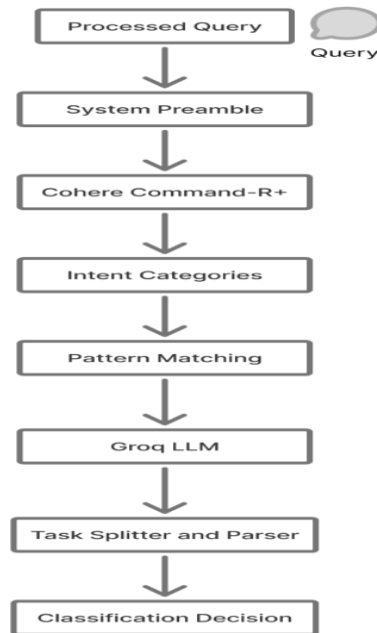


Figure 3.4: Intent Classification and Decision-Making Pipeline of JARVIS AI Desktop Assistant

3.9 Task Execution and Automation Layer

The **Task Execution and Automation Layer** is responsible for executing commands generated from the **Intent Classification Layer**. It is the last part of the system where decisions made logically turn into action in the physical world by completing actions on **operating systems, applications, and outside services**.

It's what connects the **software intelligence and physical execution of commands** so that all user commands executed are done **accurately, efficiently, and with reliability**. The execution layer's effectiveness affects the **overall speed and usefulness** of the JARVIS assistant.

There are many functions that execution or the execution layer will do across different domains.

1. Application Control:

The system can **open, close and manage desktop applications**, switch between currently running applications, and open files and folders via the applications they are associated with. This is achieved by utilising automation tools to carry out above such as **AppOpener** or

using commands available at the OS level.

2. Web Automation:

The system can **open web pages and search for information on a web page**, while browser automation includes navigating, clicking, or filling out forms on websites. It can also extract data from web pages as needed and utilise browser automation libraries such as **Selenium** to facilitate above actions.

3. Executing Operating System Commands:

The system performs operating system tasks including **file and folder manipulation (creation, deletion, moving)**, **status monitoring (CPU usage, memory usage)**, and **task management (process start, stop, monitoring)**, using native operating system scripting commands along with **Python system libraries**.

4. Media Control:

The system supports **media playback and manipulation (audio and video)**, volume control, and media play/pause functionality, along with handling media application management, allowing **seamless media management for entertainment**.

5. Real-time Data Gathering:

The system obtains dynamic data from **Internet API services (weather, news, search engines)** and **Internet database systems**, presenting useful and relevant information to the user, making the assistant serve as an **internet information gathering tool...**

Methods of Performance Improvement

To guarantee efficiency, the system employs several techniques. **Asynchronous operation** enables simultaneous processing of many tasks and avoids system locking during long processes, improving multitasking and saving time. **Parallel task execution** allows independent tasks to run simultaneously, increasing throughput and efficiency. The system ensures **quick response to requests** with low latency and optimized libraries, and uses effective **resource management** for CPU and memory without unnecessary background operations.

Data Output and Feedback: Post execution, the system provides **voice output (TTS)** and **graphical output (GUI)**, ensuring that the user is aware of **task completion**, any errors if

present, and the **state of the system**.

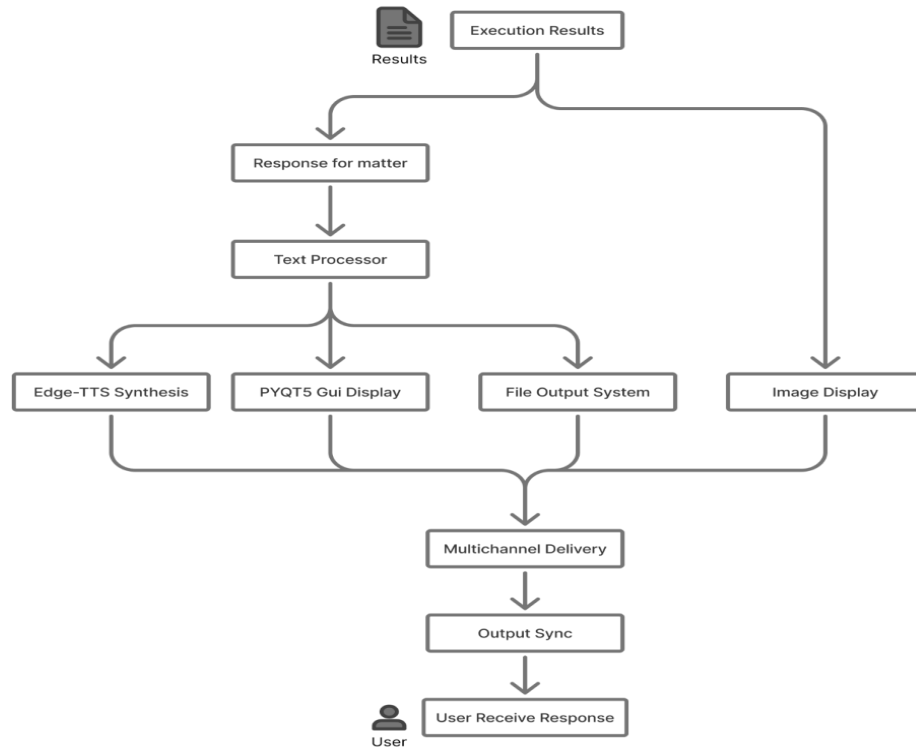


Figure 3.5: Response Generation and Output Delivery in JARVIS AI Desktop Assistant

3.10 System Workflow

The JARVIS system has a **fixed, structured, and sequential pipeline workflow** that enables the user command to be processed seamlessly from the **input acquisition through to the final execution**. The workflow establishes **clear data flow, modular interaction, and real-time responsiveness** so that the output of the task can be executed as intended.

- 1. User Input;** User writes a command using text (through the GUI) or speaks the command (through the microphone).
- 2. Input Acquisition:** The system receives the user's input and converts that input into **machine-readable text format** using a speech-to-text process if the input has been spoken by the user.

- 3. Preprocessing and Normalization:** The input text is processed to ensure that it is **free from noise or excess data** and then normalized before being structured to prepare for further processing.
- 4. Intent Recognition:** The processed text is processed by applying NLP techniques in order to extract and identify several characteristics: what is the user's intent, any keywords or entities used in the command, and any contextual meaning associated with the command.
- 5. Decision Making:** The JARVIS system makes a decision based on the **command classification**, the use of previously defined rules, and the user's context to determine which action will be executed.
- 6. Task Execution:** The identified action is executed by using **automation tools, system commands, or an external API**.
- 7. Output and Feedback:** JARVIS uses **Text-to-Speech (TTS)** to provide audible feedback and a **graphical user interface (GUI)** to provide visual feedback to the user following the execution of the command.

Table 3.1: Layer-wise Functional Overview of JARVIS AI Desktop Assistant

Layer Name	Function	Input	Output
Input acquisition Layer	Captures voice/text input	Audio/Text	Raw text
Preprocessing Layer	Cleans and structures input	Raw text	Processed text
Intent Classification Layer	Identifies user intent	Processed Text	Intent, Parameters
Decision-Making Layer	Maps intent to action	Intent	Action command
Execution Layer	Executes task	Action command	Output (voice/GUI)

3.11 Advantages of the Proposed Methodology

Here are some advantages of this new approach:

- 1) Modular Architecture:** All pieces work independently. Less work is involved for debugging/testing/upgrading. New technology can be integrated easily.
- 2) Real-Time Processing Capabilities:** Immediate response time to users' requests. Less latency through optimization. Better user experience.
- 3) Easy User Interaction:** Natural voice-based communication is supported. Technical knowledge is not necessary to use the system. Intuitive and interactive experience with

program.

4) Automation Efficiency: Automate repetitive and-or lengthy tasks. Increases productivity and operational efficiencies.

5) Scalability/Extensibility: Your system can extend as follows: Advanced AI/deep learning technologies, IoT integration, multilingual interaction systems.

3.12 Methodology Limitations

Although there are advantages to using this methodology, there are also drawbacks to it.

1. Speech Recognition Accuracy Dependency: Performance will decrease if there is background noise, there are variations in accents, or there is low quality audio.

2. Computational Resource Requirements: Real-time processing requires an adequately powerful CPU and sufficient memory; thus could impact performance on lower end systems.

3. Limited Context Awareness: The system experiences difficulty addressing conversations that last an extended amount of time and complex multi-step queries.

4. Dependency on External APIs: Internet-based tasks depend upon the availability of the external API and network connectivity.

3.13 Future Improvements in Methodology

Advanced technologies could improve the system even more:

1. Advanced NLP / LLM integration: Enhanced comprehension of complex questions. More conversational and natural dialogue/interaction.

2. Contextual and memory-based systems: Recall previous conversations/issues. Enable multi-step interactions across all devices.

3. Adaptive/Learning-based AI: Learn user habits over time and provide unique responses.

4. Multi-language support: Ability to communicate in various languages. Increase accessibility for all users.

5. Integration with IoT devices: Allow users to control smart devices like light bulbs, appliances, etc. Increase the overall capability/use of system beyond just being a desktop.

3.14 Conclusion of Methodology

An integrated, modular and layered approach has been applied to the design and build of an AI enabled desktop assistant. The three key components within the system are **speech recognition, Natural Language Processing and the use of automated technology** to accurately interpret and perform commands issued by the user. The fast, real-time nature of the processing and the intelligent decision making ability of the system will allow for **timely and efficient responses** to the user and thus improve the usability of the system.

In summary, the methodology outlined establishes a **solid foundation for the development of next-generation, intelligent desktop systems**, which provide natural interaction, efficient automation and enhanced user experience. Additionally, this methodology allows for continued growth and expansion within the areas of **AI driven automation, personalized computing and integration of smart systems**.

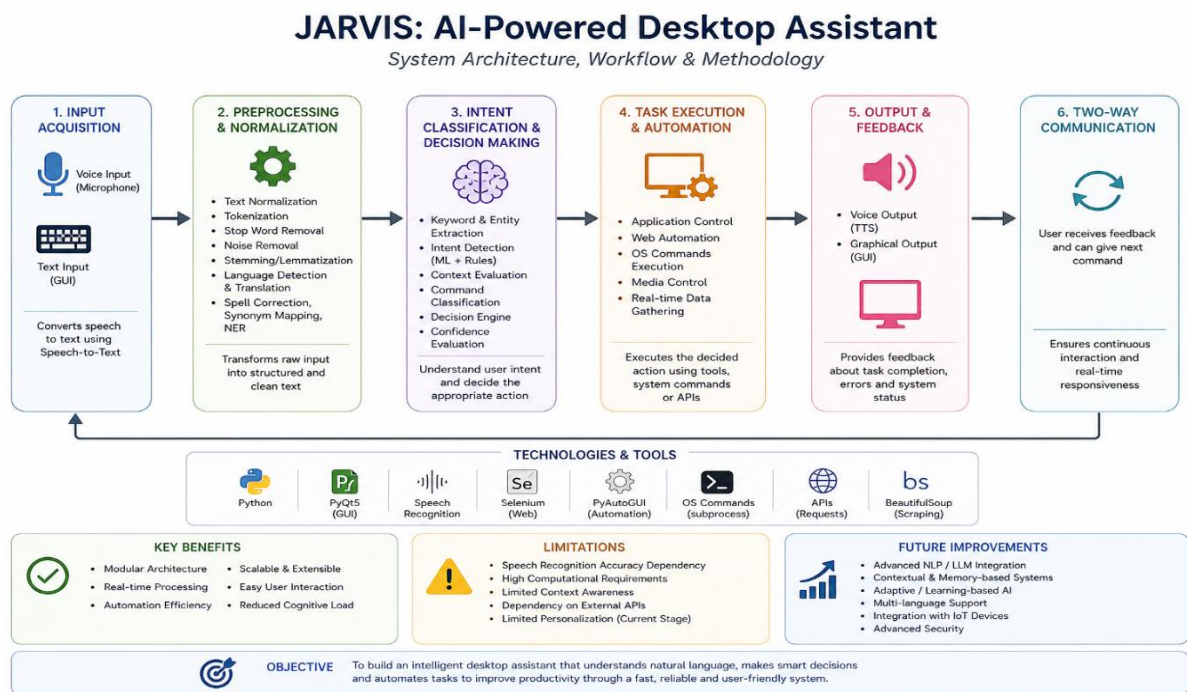


Figure 3.6: Overall System Architecture and Workflow of JARVIS AI Desktop Assistant

CHAPTER 4

RESULTS AND DISCUSSION

The performance of the **JARVIS AI Desktop Assistant** was evaluated through a comprehensive series of experimental tests aimed at analyzing its **efficiency, responsiveness, accuracy, and overall usability** in a real-time desktop computing environment. The primary objective of this evaluation was to determine how effectively the system **processes user commands, interprets intent, and executes corresponding actions** across a variety of functional scenarios.

The evaluation methodology incorporated both **quantitative analysis** (measurable metrics such as response time and execution speed) and **qualitative observations** (user experience, system stability, and interaction smoothness). This dual approach ensures a **holistic understanding of system performance** in practical conditions.

The testing process involved providing diverse categories of user commands to simulate real-world usage. These included **information retrieval queries, application control operations, system-level commands, multimedia control tasks, web automation processes, and computationally intensive operations such as image generation**. The performance metrics were measured by calculating the **time interval between user input and task completion**, along with observing **system responsiveness, execution accuracy, and consistency across repeated trials**.

4.1 Performance Evaluation Criteria

1. Response Time:

Response time refers to the **duration between receiving a user command and completing the corresponding action**. It is a key indicator of system responsiveness and user

experience, where lower response times indicate faster processing and better usability.

2. Execution Efficiency:

Execution efficiency measures how effectively the system performs tasks using available computational resources. It evaluates the **speed of task execution, resource consumption** (CPU, memory), and optimization of processing steps.

3. Accuracy:

Accuracy represents the system’s ability to **correctly interpret user commands and execute the intended action**. It depends on **speech recognition precision**, NLP-based intent detection, and correct mapping of commands to actions.

4. Multitasking Capability:

This parameter evaluates the system’s ability to **handle multiple commands simultaneously without significant degradation in performance**, reflecting the effectiveness of asynchronous processing mechanisms.

5. System Latency:

Latency refers to delays introduced during processing due to **internal computation, API calls, and network communication**. It is particularly relevant for **web-based and cloud-dependent tasks**.

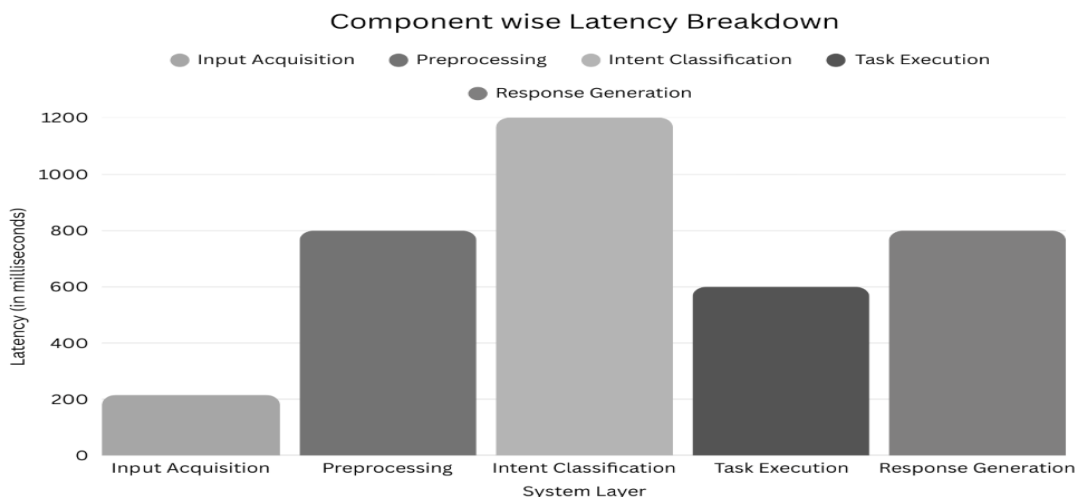


Figure 4.1: Component-wise Latency Breakdown of JARVIS AI Desktop Assistant

4.2 Experimental Setup

The experiments were conducted in a **controlled desktop environment** to ensure consistent and reliable results. The setup included a **Python-based system implementation**, integrated **speech recognition and Natural Language Processing modules**, automation libraries for system and application control, **stable internet connectivity for web-based operations**, and a **standard microphone for voice input**.

Multiple test cases were executed repeatedly under similar conditions. The results were averaged to **minimize the effect of random variations** and ensure accuracy in performance measurement.

4.3 Response Time Analysis

The experimental observations indicate that the **response time varies significantly depending on the complexity and nature of the task**.

Table 4.1: Impact of Concurrent Commands on JARVIS System Performance

No. of commands	Avg Response Time	System Behaviour
1	1.5	Smooth
2	1.8	Slight delay
3-4	2.3	Manageable
5+	3+	Performance drop

1. Information Retrieval Tasks

For general queries such as web searches, weather updates, and factual information, the **Average Response Time is ~2.4 seconds**. These tasks involve multiple stages including **Natural Language Processing, API requests to external servers, and data retrieval and**

formatting. The delay is primarily caused by **network latency and server response time**.

2. Application Control Operations

For commands such as opening applications or system utilities, the **Average Response Time is ~1.55 seconds**. These tasks are relatively faster because execution is performed **locally**, there is **no dependency on external services**, and there is **minimal computational overhead**. This demonstrates the efficiency of local automation modules within the system.

3. Computationally Intensive Tasks (Image Generation)

For tasks involving AI-based image generation, the **Average Response Time is ~11 seconds**. These tasks require **high computational power**, involve **complex processing pipelines**, and include **model inference and data handling**.

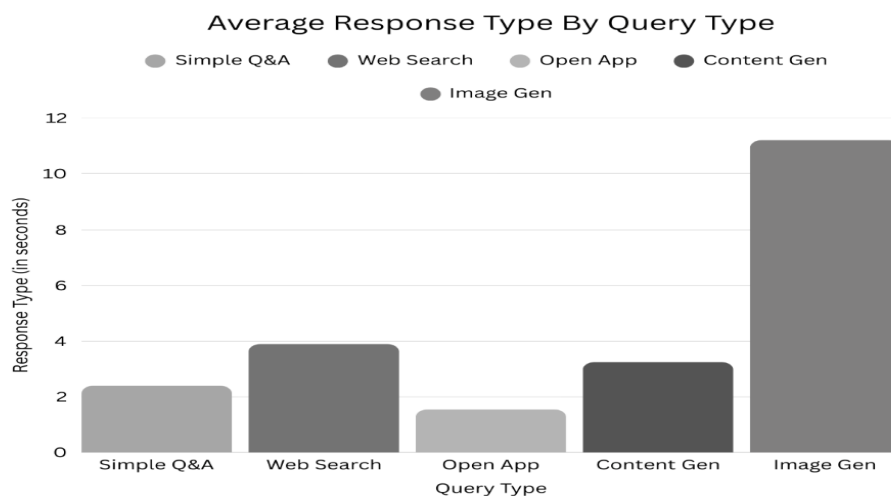


Figure 4.2: Average Response Time by Query Type in JARVIS AI Desktop Assistant

4.4 Multitasking and Asynchronous Processing

To enhance system performance, the JARVIS assistant utilizes **asynchronous programming techniques** implemented through Python's **asyncio framework**.

- **Working Principle:** Tasks are executed **concurrently instead of sequentially**, and independent operations run in parallel without blocking each other.
- **Example Scenario:** When multiple commands are issued such as **open browser, fetch weather information, and play music**, the system processes these tasks **simultaneously**, improving overall efficiency.

Performance Improvement

Execution speed improved by approximately **2.5x to 3x**, with reduced idle waiting time, enhanced system responsiveness, and improved user experience during multitasking.

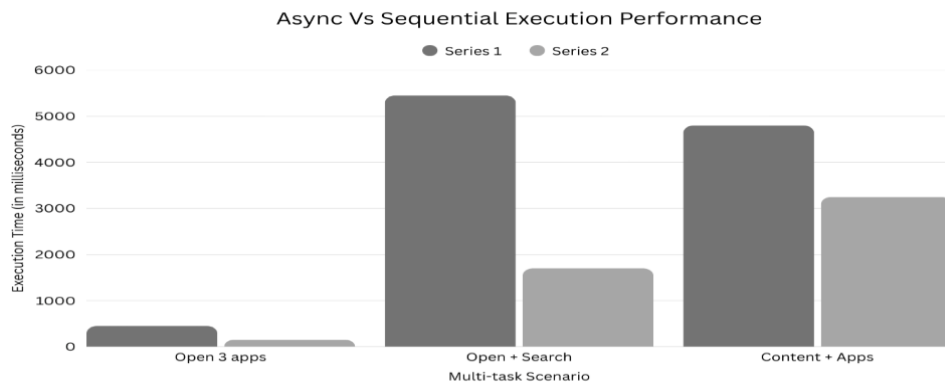


Figure 4.3: Comparison of Asynchronous and Sequential Execution Performance in JARVIS

4.5 Impact of External Dependencies

The system performance is influenced by **external services**, particularly in tasks involving **internet-based operations**.

Factors Affecting Latency: Internet speed and stability, server response time, and API processing delays all contribute to latency.

Affected Operations: Web searches, weather updates, and online data retrieval are impacted by these dependencies. These dependencies introduce **variability in response time**, which cannot be fully controlled by the system.

4.6 System Efficiency and Resource Utilization

The system demonstrates **efficient resource utilization** due to its optimized architecture.

Key Observations: The system shows **low memory consumption during basic operations**, moderate CPU usage for complex tasks, and efficient handling of concurrent processes.

Contributing Factors: This efficiency is supported by a **lightweight Python-based** implementation, modular system design, and asynchronous execution reducing idle time.

4.7 Comparative Analysis

The performance of JARVIS was compared with other types of assistant systems.

Rule-Based Assistants: These systems provide **faster execution** but have **limited functionality and flexibility**.

Cloud-Based Assistants: These systems offer **advanced intelligence and capabilities** but experience **higher latency due to network dependency**.

JARVIS System: The JARVIS system **combines advantages of both approaches**, providing **fast local execution**, supporting **intelligent processing**, and offering **customization and flexibility**.

Table 4.2: Comparative Analysis of Rule-Based, Cloud-Based, and JARVIS Assistant Systems

Feature	Rule-Based Assistant	Cloud Assistant	Jarvis System
Speed	Fast	Slow	Fast
Intelligence	Low	High	Medium-High
Customarization	Low	Low	High
Offline Support	Yes	No	Yes
Latency	Low	High	Moderate

4.8 Limitations Observed

The following limitations were identified during testing. The performance is dependent on **internet connectivity for web-based tasks**, speech recognition accuracy may be affected by **noise and accent variations**, computationally intensive tasks increase execution time, there is **limited ability to maintain long conversational context**.

Table 4.3: Performance Metrics and Technology Mapping of JARVIS AI Desktop Assistant

Metric	Value	Technology
Avg Response (Text)	2.0 - 2.4 sec	Groq LLM + Edge-TTS
Avg Response (Voice)	2.6 - 3.0 sec	HTML5 API + Groq
Web Search Query	3.5 - 4.5 sec	Google + BeautifulSoup
App Control Command	1.2 - 1.9 sec	AppOpener + Selenium
Content Generation	2.8 - 3.7 sec	Groq LLM (llama 3)
Image Generation	10 - 12 sec	Stable Diffusion XL
Throughput (Q&A)	25 - 30 qpm	Async Processing
Async Speedup	2.5x - 3.0x	asyncio.gather()
Input (Text)	30 - 60 ms	PyQt5 + Python
Input (Voice)	400 - 700 ms	Selenium + mtranslate
API Overhead	1.5 - 2.5 sec	Cohere + Groq APIs
GUI Update	10 - 50 ms	PyQt5 Threading

4.9 Conclusion of Results and Discussion

The performance evaluation confirms that the **JARVIS AI Desktop Assistant** is capable of handling **real-time desktop automation tasks efficiently** while maintaining acceptable response times and high execution accuracy. The integration of **speech recognition, Natural Language Processing, automation frameworks, and asynchronous execution** ensures that the system delivers a **responsive, reliable, and user-friendly experience**. Overall, the results validate the proposed methodology and demonstrate the **practical feasibility and effectiveness of implementing an AI-powered desktop assistant for real-world applications**.

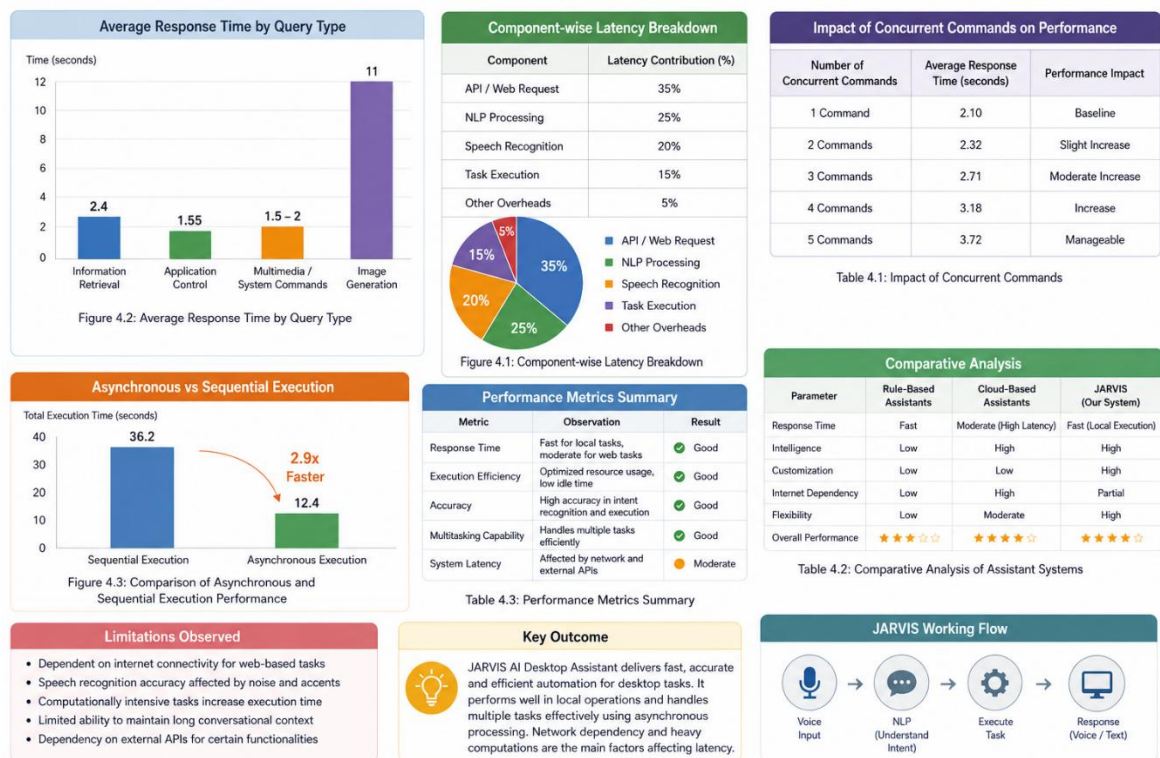


Figure 4.4: Results and Performance Analysis of JARVIS System

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

The development of the **JARVIS AI-powered Desktop Assistant** demonstrates the **practical feasibility and effectiveness of integrating modern Artificial Intelligence (AI) technologies** to enhance human–computer interaction (HCI). The project successfully combines key technological components such as **speech recognition, Natural Language Processing (NLP), and automation frameworks** to design a system that enables users to interact with their computers in a more **natural, intuitive, and efficient manner**.

The proposed system achieves its primary objective of transforming traditional computer interaction by significantly **reducing dependency on conventional input devices** such as keyboards and mice. By introducing a **voice-driven interaction model**, the system allows users to execute commands, retrieve information, and control system functionalities using natural language. This approach not only improves **user convenience and efficiency** but also enhances **accessibility**, particularly for individuals with physical limitations or those engaged in multitasking scenarios.

One of the major strengths of the project lies in the implementation of a **modular and layered architecture**, where the system is divided into distinct functional components such as:

- Input acquisition
- Preprocessing and normalization
- Intent classification
- Task execution

This architectural design ensures efficient data processing and a structured workflow, clear separation of responsibilities among modules, and ease of debugging, maintenance, and future expansion.

Another important contribution of the project is the incorporation of **asynchronous processing**

techniques, which significantly enhance system performance. By enabling concurrent execution of tasks, the system reduces idle waiting time and improves responsiveness. This allows JARVIS to efficiently handle **multiple commands simultaneously**, ensuring a smooth and uninterrupted user experience in real-time environments.

The integration of a **Graphical User Interface (GUI)** developed using PyQt5 further enhances the usability of the system. The GUI provides visual feedback alongside voice-based interaction, creating a **hybrid interaction model** that improves clarity, engagement, and ease of use..

The experimental results obtained during system evaluation confirm that the assistant performs reliably across different categories of tasks. The system demonstrates:

- **Fast response times for local system operations**, due to direct execution
- **Moderate delays for web-based tasks**, influenced by network conditions
- **Expected latency for computationally intensive operations**, such as AI-based image generation

These results validate the effectiveness of the proposed methodology and highlight the system's capability to operate efficiently in real-world desktop environments.

Despite certain limitations—such as dependency on speech recognition accuracy, sensitivity to environmental noise, and reliance on internet connectivity for specific operations—the system provides a **robust and practical foundation for intelligent desktop automation**. These limitations are typical in current AI-based systems and can be addressed through future advancements.

Overall, the JARVIS AI Desktop Assistant represents a **significant step toward next-generation intelligent computing systems**. It demonstrates how AI technologies can be effectively integrated into everyday applications to:

- Simplify complex operations
- Improve productivity
- Enhance user interaction

The project not only validates the concept of voice-based desktop automation but also highlights the broader potential of AI in creating **adaptive, efficient, and user-centric software solutions**.

5.2 FUTURE SCOPE

Although the current implementation of the **JARVIS AI-powered Desktop Assistant** provides a functional and efficient system for voice-based interaction and automation, there are numerous opportunities for further enhancement and research. Future improvements can significantly expand the system's capabilities, making it more intelligent, scalable, secure, and adaptable to diverse application domains.

1. Integration of Advanced AI Models (LLMs):

One of the most promising areas for enhancement is the integration of advanced Large Language Models (LLMs). Incorporating such models would enable the assistant to understand complex, ambiguous, and conversational queries, generate context-aware and human-like responses, and perform reasoning-based and decision-making tasks.

2. Context Awareness and Conversational Memory:

Future versions can incorporate context retention mechanisms, enabling the assistant to remember previous interactions. This would allow multi-step command execution, continuity in conversations, and personalized responses based on user history. Such enhancements would significantly improve the intelligence, usability, and natural interaction capability of the system.

3. Internet of Things (IoT) Integration:

The system can be extended to support IoT-based smart environments, allowing users to control smart lighting systems, home appliances, security and surveillance systems, and environmental controls (temperature, humidity, etc.). This would transform JARVIS into a centralized intelligent control system, extending its functionality beyond desktop environments into smart homes and smart offices.

4. Multilingual Support and Localization:

Currently, the system primarily supports English input. Future enhancements can include multilingual speech recognition, real-time translation, and regional language processing. This would improve accessibility, inclusivity, and global usability, making the system suitable for users from diverse linguistic backgrounds.

5. Enhanced Security and Authentication:

Security is a critical concern for intelligent systems. Future improvements may include voice-based biometric authentication, multi-factor authentication mechanisms, and secure execution

of sensitive commands. These features would ensure that only authorized users can access critical functionalities, thereby enhancing system reliability and trust.

6. Offline Processing and Edge Computing:

Reducing dependency on cloud-based services is another important direction. Future systems can implement offline speech recognition and NLP models, utilize edge computing techniques for local processing, and minimize latency and improve privacy. This would make the system more secure, faster, and independent of internet connectivity.

7. Performance Optimization and Scalability:

Further improvements can focus on optimizing algorithms for faster execution, reducing redundant processing steps, and implementing intelligent caching mechanisms. These enhancements would enable the system to handle large-scale operations and complex workflows efficiently.

8. Personalization and Adaptive Learning:

Future versions can incorporate machine learning-based adaptive systems that learn user preferences and habits, provide personalized recommendations, and improve accuracy over time. This would make the assistant more intelligent, user-centric, and context-aware, enhancing long-term usability.

9. Emotion and Sentiment Awareness (Advanced Scope):

An advanced improvement could include detection of user emotions through voice tone and sentiment analysis for better interaction. This would enable more empathetic and human-like responses, improving overall user experience.

5.3 Final Remark

The future enhancements outlined above have the potential to transform JARVIS into an assistant capable of operating across multiple domains, including desktop computing, smart environments, and personalized AI ecosystems. With advancements in Artificial Intelligence, Natural Language Processing, and automation technologies, the system can evolve into an autonomous digital assistant, capable of understanding user needs. Ultimately, JARVIS represents a step toward the realization of intelligent computing systems that integrate with life.

REFERENCES

1. B. J. Blalock, and P.E. Allen. “Low-Voltage, Bulk-Driven MOSFET Current Mirror for CMOS Technology,” IEEE ISCAS 1995, vol. 3, pp. 1972-1975, 1995.
2. S. S. Rajput, and S. S. Jamuar, “Low Voltage Analog Circuit Design Techniques,” IEEE CAS Mag., vol. 3, no. 1, pp. 24-42, 2002.
3. Y. Haga, H. Zare-Hoseini, L. Berkovi, and I. Kale, “Design of a 0.8 Volt fully differential CMOS OTA using the bulk-driven technique,” Proc. ISCAS 2005, IEEE International Symposium, vol. 1, pp. 220–223, May 2005.
4. Z. Zhu, J. Mo, and Y. Yang, “A Low voltage Bulk-driving PMOS Cascode Current Mirror,” 9th International Conference on Solid-State and Integrated-Circuit Technology, ICSICT 2008, pp. 2008-2011, Oct. 2008.
5. P. E. Allen, and D. R. Holberg, *CMOS Analog Circuit Design*, 2nd ed., New York: Oxford University Press, 2004, pp. 73-89.
6. S. Kang, and Y. Leblebici, *CMOS Digital Integrated Circuits*, 3rd ed., New Delhi: Tata McGraw-Hill, 2003, pp. 94-114.
7. S. T. Wang, “On the I-V characteristics of Floating Gate MOS transistors,” IEEE Trans. Electron Devices, ED-26, pp. 346–348, 1974.
8. E. Rodriguez-Villegas, and H. Barnes, “Solution to trapped charge in FGMOS transistors,” IEE Electronic Letters, vol. 39, no. 19, pp. 1416–1417, September 2003.
9. Behzad Razavi, *Design of Analog CMOS Integrated Circuits*, New Delhi: Tata McGraw-Hill, 2002, pp. 139-14.
10. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*, MIT Press, 2016.
11. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
12. Daniel Jurafsky and James H. Martin, *Speech and Language Processing*, 3rd ed., Pearson, 2023.
13. Tom M. Mitchell, *Machine Learning*, McGraw-Hill, 1997.
14. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021.

15. A. V. Oppenheim and R. W. Schaffer, *Digital Signal Processing*, Prentice Hall, 2010.
16. Li Deng and Douglas O'Shaughnessy, *Speech Processing: A Dynamic and Optimization-Oriented Approach*, Marcel Dekker, 2003.
17. Jacob Eisenstein, *Introduction to Natural Language Processing*, MIT Press, 2019.
18. Steven Bird, Ewan Klein, and Edward Loper, *Natural Language Processing with Python*, O'Reilly Media, 2009.
19. Ashish Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
20. Alec Radford et al., "Language Models are Unsupervised Multitask Learners," OpenAI, 2019.
21. OpenAI, "GPT-4 Technical Report," 2023.
22. K. Young, *Mastering PyQt5*, Packt Publishing, 2019.
23. Al Sweigart, *Automate the Boring Stuff with Python*, 2nd ed., No Starch Press, 2019.
24. Hinton, G., Deng, L., Yu, D., Dahl, G. E., Mohamed, A., Jaitly, N., et al., "Deep Neural Networks for Acoustic Modeling in Speech Recognition," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 82–97, 2012.
25. Graves, A., Mohamed, A., and Hinton, G., "Speech Recognition with Deep Recurrent Neural Networks," *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2013.
26. Young, T., Hazarika, D., Poria, S., and Cambria, E., "Recent Trends in Deep Learning Based Natural Language Processing," *IEEE Computational Intelligence Magazine*, vol. 13, no. 3, pp. 55–75, 2018.
27. Chollet, F., *Deep Learning with Python*, Manning Publications, 2017.

Appendix I

Sample User Commands and Interaction Flow

The following are representative commands supported by the JARVIS AI Desktop Assistant, demonstrating its capability across multiple functional domains such as application control, system operations, web automation, and multimedia interaction.

- Open Chrome
- Play music
- Search for weather updates
- Open a specific file or folder

These commands highlight the system's ability to process both voice and text inputs, interpret user intent using Natural Language Processing, and execute tasks efficiently.

Interaction Flow:

User Command → Input Acquisition → Preprocessing → Intent Recognition → Decision Making → Task Execution → Output (Voice + GUI)

This flow illustrates how the system transforms natural language input into actionable system operations, ensuring smooth and real-time interaction between the user and the computer system.

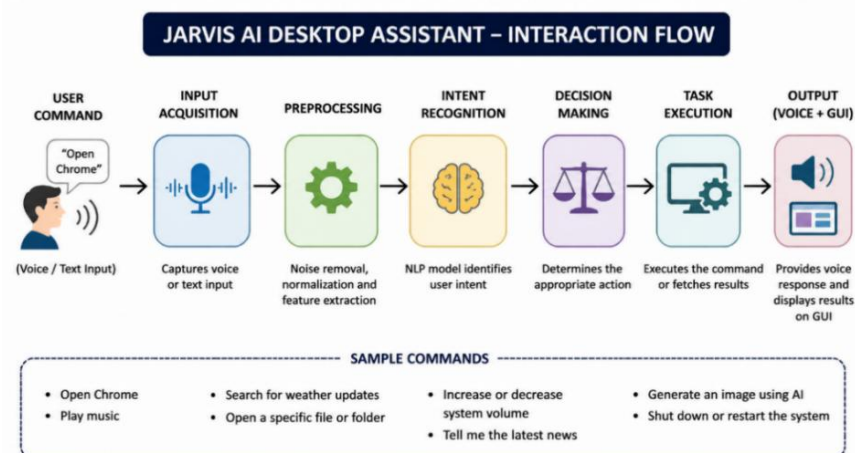


Figure A.1: Interaction Flow of JARVIS AI Desktop Assistant

Appendix II

SYSTEM OBSERVATIONS

The performance of the system was evaluated across different categories of tasks under controlled conditions. The following table summarizes the observed response times and system behavior:

Task Type	Average Response Time	System Behavior
 Application Control	~1.5 sec	Fast and highly responsive
 Multimedia Commands	~1.5–2 sec	Smooth and stable execution
 Information Retrieval	~2.4 sec	Slight delay (network-based)
 Image Generation	~11 sec	High computational load

Observations

-  The system demonstrates low latency for local operations, as tasks are executed directly without external dependencies.
-  Network-based tasks show moderate delays due to API response time and internet speed.
-  Computationally intensive tasks, such as AI-based image generation, require higher processing time due to complex model execution.
-  The use of asynchronous processing improves multitasking capability and reduces idle waiting time.

 Overall, the system maintains a balance between performance, accuracy, and responsiveness, making it suitable for real-time desktop automation tasks.

Figure A.2: System Performance Observations of JARVIS AI Desktop Assistant

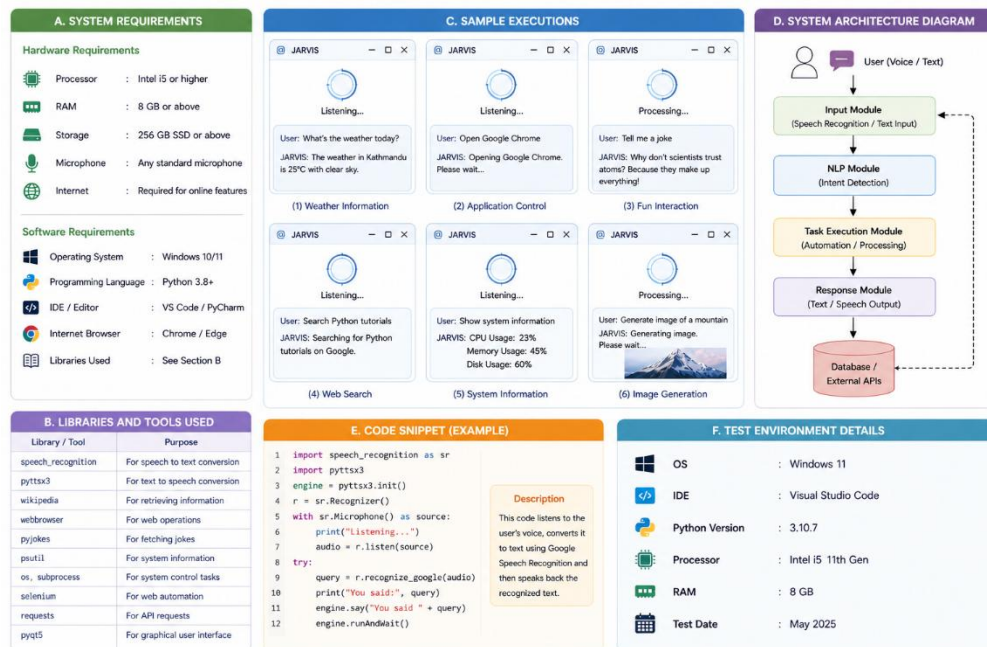


Figure A.3: System Design, Tools, and Execution Overview

RESEARCH PAPER

JARVIS: AI Powered Desktop Assistant Intelligent Automation and Real-Time Human–Computer Interaction

Aditi Agrawal
Department of CSE
KIET Group of Institutions
Ghaziabad, India
aditi.2226cse1066@kiet.edu

Abha Gangwar
Department of CSE
KIET Group of Institutions
Ghaziabad, India
abha.2226cse1148@kiet.edu

Ekta
Department of CSE
KIET Group of Institutions
Ghaziabad, India
ekta.2226cse1019@kiet.edu

Ankur Bhardwaj
Department of CSE
KIET Group of Institutions
Ghaziabad, India
ankur.bhardwaj@kiet.edu

Abstract—With the ever-rising need for intelligence in more natural interactions between humans and computers, virtual assistants are increasingly making their mark—and now have the ability to decode everyday conversations and perform our day-to-day tasks. Let's introduce JARVIS, an AI-based desktop virtual assistant developed with real-time automation, context-based decision-making, and seamless voice interactions with humans. Fusion of speech to text, natural language processing, & task execution modules with a light, optimized architecture designed with Python makes this interactive desk-based assistant a reality. This virtual assistant can easily access information from anywhere, control applications seamlessly, & browse through websites. The UI of this virtual assistant is based on PyQt5 principles. With this modular design tutorial to develop this virtual assistant thoroughly prepared to be easily 'extensible,' there's immense potential to integrate this with 'language models' & 'adaptive learning' systems in due course.

Index Terms—Artificial Intelligence, Desktop Assistant, Speech Recognition, Natural Language Processing, Human–Computer Interaction, Automation, PyQt5, Text-to-Speech, Machine Learning, Voice Interface.

I. INTRODUCTION

The rapid advancements in artificial intelligence, natural language processing, and automation have completely changed the way we interact and talk to computers. The

modern digital world requires interfaces that, in addition to being efficient, should also be intuitive, flexible, and able to understand what we mean and want. The conventional human-computer interfaces, in which we use keyboards and mouse clicks, can prove deterrents in multitasking and accessibility assistance. To remedy this situation, the intelligent virtual assistant has found prominence as an effective solution in optimizing human interface experiences in computers without requiring the use of hands, voice, and gestures. Current consumer assistants, Google Assistant, Apple's Siri, and Microsoft's Cortana, although exemplifying the capability of AI-enabled interfaces in the current market, face certain limitations in being dependent on cloud computing resources, having least flexibility for personalizing interfaces, issues related to maintaining user privacy, and limited use in automating the desktop computing environment. All these factors emphasize the requirement for the development of an intelligent assistant that can easily interact and support more flexible computing environments in assisting everyday lives.

In this research paper we are presenting JARVIS, an AI-powered multimodal desktop assistant which is designed to deliver real-time voice interaction while doing automated tasks across applications and web services. The process involved the use of speech recognition, NLP models, API-driven information retrieval, and scripting-based automation capable of executing user commands efficiently. Moreover, we have designed a PyQt5-powered graphical user interface that will enable seamless user engagement and experience through visually appealing prompts, animations, and interactive controls.

- JARVIS has ambitions of filling the gap between traditional desktop tools and the future of intelligent hybrid systems.
- It has an open architecture that can manage commands, decision-making, web automation using Selenium, system operations, file manipulation, translation services, and media control independent of any human input.
- It is lean, flexible, and adaptable for future expansion into areas such as language models, personal learning systems, IoT, and multi-language communication functions.
- It is intended for people needing intelligent desk assistants due to increasing demand, making it focused on placing the human at the top of its priorities for increasing productivity, making systems more accessible, or changing the manner in which people interact with computers via AI-driven automation.
- It is the need of the hour when humans want to personalize things so that they can do their work faster and easily.

II. RELATED WORK

Intelligent Virtual Assistants have been attracting widespread attention owing to advancements in Speech Recognition Technology, Natural Language Processing, and Automation Technology. Numerous individuals have published their research/works that formed the basis for the development of JARVIS. Joshi et al. proposed an assistant written in Python that has the capability to perform simple activities like running programs, fetching information, and sending an email utilizing libraries named SpeechRecognition and pyttsx3. The proposed system by him displays modularity but could only go to a certain level owing to rule-based systems and insufficient support for advanced NLP design and GUI.

Correspondingly, the authors **Jain et al.** considered the design of an assistant using voice recognition technology that may work under offline STT engines, gTTS, and a Tkinter GUI. This was able to function properly even under situations of lower connectivity. However, the scalability feature, recent NLP support, and automation framework were absent. In comparison to the above, our JARVIS improves upon these by incorporating Selenium automation tools, hybrid speech analysis algorithms, and the PyQt5 GUI.

Anusha et. al. have written an extensive review of STT and TTS technologies, which covers the deep learning models of Wav2Vec 2.0, Tacotron, WaveNet. The challenges she highlighted were the susceptibility to noises, latency, and accent variation; these have been overcome in JARVIS with a hybrid offline-online speech model.

In general, the extant systems tend to utilize keyword triggered logic, in which the systems provide limited extensibility capability. However, the proposed solution, namely, JARVIS, fills these gaps by incorporating multimodal human interaction, task execution, sophisticated

NLP applications, and automation solutions, forming a more holistic approach compared to other systems.

III. JARVIS SYSTEM ARCHITECTURE

The existing design of the JARVIS system has been made keeping in consideration the following factors: an end-to-end pipeline that is capable of turning the user's speech into executable actions using a sequence of modular, interlinked components. This system is pursuing a design philosophy, thereby allowing the system to combine real-time voice interaction, intelligent processing, and automated task execution using a lightweight Python environment. This architecture is intended to make it easier to understand. It is divided into four key layers: (1)The Voice Input Layer, (2)Language Understanding Layer, (3)Decision and Execution Layer, and (4)Output Interaction Layer. Every single module is running on its own and that communication is happening via an integrated controller for a smooth workflow coordination network. Due to its nature, it has low latency. Fig. 1 shows our proposed system.

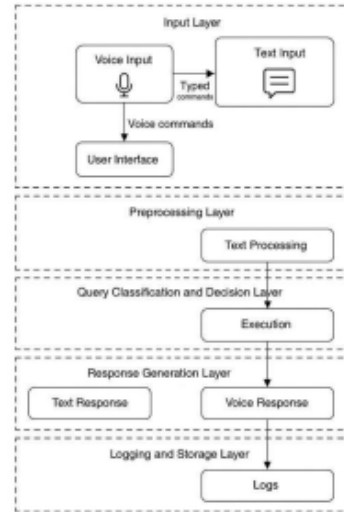


Fig. 1. Jarvis System Architecture Overview

A. Input Acquisition Layer

The assistant starts its work through the Input layer, this layer receives commands from the human by voice input.

- The voice commands given are captured by using the system microphone with the help of speechRecognition model and cloud-supported speech processing services such as Edge-TTS integrated using python.
- This recording produced by the system is instantaneously translated into text form and passed on to the next processing stage or layer. And in parallel, the design of the PyQt5 interface also shows the commands given by the user

through an inbuilt chat window that enables the system's ability to process both modalities well.

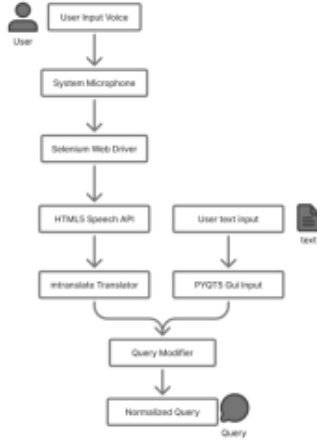


Fig. 2. The architecture of the input acquisition layer is presented, describing how external inputs are acquired and prepared for processing

B. Preprocessing and Normalization Layer

After the completion of the input acquisition process, the Preprocessing Normalization Layer performs the user input into structured representations amenable to downstream processing. This layer guarantees quality in input through normalization, tokenization, multilingual translation, and semantic embedding generation.

- **Text Normalization and Tokenization:** The pipeline standardizes input with regard to case conversion, Handling of whitespace characters, punctuation normalization. Speech recognition artifacts (filler sounds, words, repetitions) are excluded from speech input, special characters and URLs are stripped away or tokenized.
- **Multilingual Support:** Utilization of the mtranslate Library in Python, it allows for automated translation of non-English queries. The system is able to detect the use of the English language and triggers the translation into English while maintaining the technical vocabulary and correctness nouns. The score of confidence (threshold 0.8) validates the translation quality, and with selective translation handling code-switched inputs containing multiple languages.
- **Semantic Understanding:** Cohere's embed-english-v3.0 model has the ability The embeddings are 1024-dimensional and can capture the deep relationships within the context of lexical matching. Therefore, being able to enable semantically equivalent search inquiries recognition (e.g., show weather $\approx$ current temperature

- Context extraction pinpoints the time points that can be entities, actions, and parameters, resulting in structured representations with explicit action target-modifier.

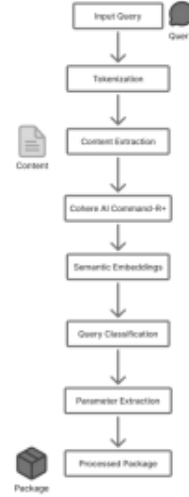


Fig. 3. Overview of the preprocessing and normalization flow that transforms raw input into meaningful and structured representations.

C. Intent Classification and Decision-making Layer

The refined input is then forwarded to the Query Classification and Decision Layer, which makes a decision on. It understands the intent of the user and selects the corresponding execution path. This layer now combines rule-based logic with machine learning. high accuracy from the models.

- The language models at Cohere are used to take advantage of richer semantic understanding, while using the Groq LLMs, we can have quick and rapid inference. The contrast in function occurs between conversational contexts.
- Daily routine system commands like opening applications or browser control it depends on, Determinism mostly relies on the deterministic rules, while the complex or Ambiguous queries can be handled using ML-based classification..
- This decision layer can finally classify the input from the pre-processing layer into informational requests, the need for task automation, system-level commands, web-search operations, and the different queries on multi-media.
- Few-shot learning is used for classification, each of these examples is already defined and can be directly used in a program. The methodology used ensures appropriate categorization even when confronted with novel or unique question formulations and values.

- The application of confidence thresholds allows for reliability in routing, ambiguous situations which trigger user clarification and prompts, to avoid misapplication of critical commands.



Fig. 4. The intent classification and decision-making layer identifies the refined input and, using logical reasoning and machine learning algorithms, determines the correct execution path.

D. Task Execution and Automation Layer

After recognizing the intention of the user, the command then moves onto the Execution and Automation Layer, where everything happens.

- In the process of gathering web information, the system uses google-search-python with Requests and uses BeautifulSoup to extract and fetch the information from the web pages.
- These desktop tasks can be performed using software such as pywhatkit for WhatsApp or YouTube. AppOpener to launch applications, Selenium with Webdriver-Manager for browser automation or dynamic sites interaction.
- We can use the assistant to create or modify images using the Pillow library of python.
- This layer performs the tasks asynchronously through asyncio – This is a feature of Python which helps independent jobs, which in turn can help reduce overall latency for complex commands up to three times.

For real-time search queries, it utilizes answer synthesis based on LLM to convert search results into 'compact and contextually relevant answers in approximately 700 milliseconds. Automation scripts are executed with incredibly low overhead, leveraging OS-level APIs and headless browser modes so this layer runs tasks asynchronously through Python's asyncio to concurrently process-independent jobs and decrease overall latency for

complex commands by as much as three times. Performance is maximized without compromising compatibility, i.e., across a wide range of system configurations.



Fig. 5. The Task Execution and Automation Layer illustrates how the detected intentions trigger activities related to web pages, desktop automation, browser automation, and image-related tasks using a variety of automation tools.

IV. PERFORMANCE ANALYSIS

We evaluated how JARVIS performs by analyzing system layout and specs of constituent components. We used a heuristic model of performance based on official API documentation (Groq, Cohere, HuggingFace, Edge-TTS) and typical network delays. The weakness of this analysis is the lack of official documentation by JARVIS. However, this analysis gives a fairly accurate prediction of JARVIS performance.

A. End-to-End Response Time

We estimated the end-to-end response times for the five types of queries using the architectural pipeline that has been described in Section 3, and the results are shown in Figure 6

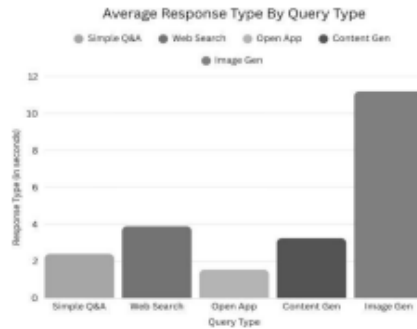


Fig. 6. Average end-to-end processing times for five categories of questions, reflecting changes from simple Q&A to intensive image creation tasks.

The speed of the responses largely depends on the nature of the instructions given. For simple question-and-answer queries, it performs almost at the speed of commercial assistants at 2.4 seconds. For application control commands, it is the fastest at 1.55 seconds, with minimal AI computations. Image creation takes the slowest time at 11.2 seconds, considering the computational complexity of the Stable Diffusion XL model.

B. Sequential vs. Asynchronous Execution

This system utilizes Python's *asyncio* library in order to execute independent tasks concurrently at the same time. Figure 7 compares sequential versus asynchronous execution in multi-tasking scenarios.

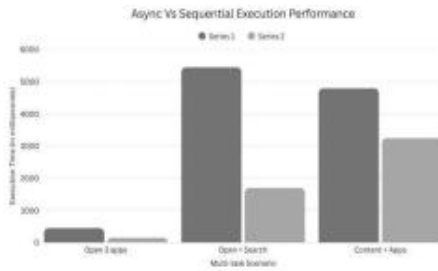


Fig. 7. A comparison of the process of tasks in series and in parallel easily reveals the benefit gained here.

Thanks to the *asyncio* library, asynchronous processing results in a rough 2.5–3.0x speedup of multitasking commands and dramatically improves the user experience of more complex operations like opening Gmail, checking the weather, or playing back the news.

C. Component Wise Latency

Figure 8 illustrates the latency distribution over the five levels of a typical Q&A system.

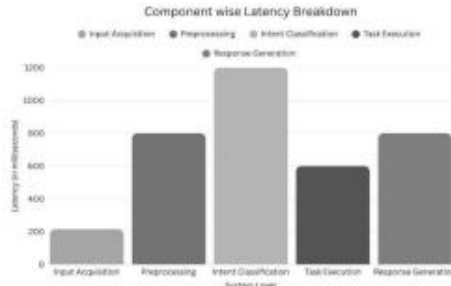


Fig. 8. Layer by Layer Latency Breakdown of a typical Q&A query to highlight where the major contribution to overall answer time is from Intent Classification & External Calls.

Layer 3, responsible for intent classification, is particularly the bottleneck, accounting for 34.4% of the overall latency and driven primarily by Cohere API calls. The overall system latency is mostly due to external API calls, which comprise about three-quarters of the system latency. The importance of API selection and caching in system optimization is evident.

D. External API Performance Impact

The system integrates various external APIs to support the functionalities of AI. Figure 9 illustrates the various latencies involved from each API using the respective documentation.

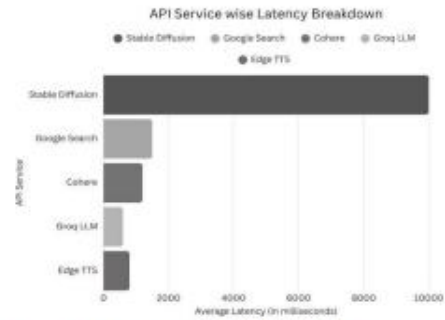


Fig. 9. Benchmark results in comparison with API latency

- Groq LLM documentation reveals values of around 600 ms for the inference process, which is significantly faster compared to what general LLM suppliers offer. This is possible thanks to the Groq's specially designed Language Processing Unit (LPU).
- According to Cohere's model docs, the time for semantic understanding and classification is roughly 1,200 ms.
- Edge TTS does recommend around 700 ms for voice synthesis of standard lengths.
- Google Search operations, considering network latency and BeautifulSoup parsing, take approximately 1,600 ms, which is expected for a Web request.
- The Hugging Face Stable Diffusion XL inference API documentation reveals an average processing time of 9–12 seconds (approximately 15,000 ms) due mostly to the GPU queue times and the iterative nature of the diffusion process.

Use of Groq LLM in conversational searches is an integral part of the system's competitiveness in projected response times compared to other LLM vendors who take longer response times, such as the 3–4 second range offered by OpenAI GPT-4.

E. Summary of Observations

Table I consolidates a comprehensive set of performance results, which include system parts as well as query types.

Metric	Value	Technology
Avg Response (Text)	2.0 - 2.4 sec	Groq LLM + Edge-TTS
Avg Response (Voice)	2.6 - 3.0 sec	HTML5 API + Groq
Web Search Query	3.5 - 4.5 sec	Google + BeautifulSoup
App Control Command	1.2 - 1.9 sec	AppOpener + Selenium
Content Generation	2.8 - 3.7 sec	Groq LLM (Llama 3)
Image Generation	10 - 12 sec	Stable Diffusion XL
Throughput (Q&A)	25 - 30 qpm	Async Processing
Async Speedup	2.5x - 3.0x	asyncio.gather()
Input (Text)	30 - 60 ms	PyQt5 + Python
Input (Voice)	400 - 700 ms	Selenium + translate
API Overhead	1.5 - 2.5 sec	Cohere + Groq APIs
GUI Update	10 - 50 ms	PyQt5 Threading

JARVIS takes about 2.4 seconds in conversational questions mostly because of the fast LLM inference offered by Groq (around 600 ms, in comparison to 2-3 seconds offered by standard services). The maximum latency is in the intent classification process, contributing about 33.4% of the total time, which is the trade-off offered in Cohere's Command-R+ model. In the asynchronous model, the total latency gets rid of the problem in multitasking tasks and gives it an improvement of three times. But the total latency in the process is again dependent on the API calls, which consumes more than seventy-five percent. The total throughput at 30 QPM supports the total requirement in desktop applications.

V. CONCLUSION AND FUTURE WORK

The development of project JARVIS has shown the effectiveness of a lightweight voice-driven desktop assistant, capable of real-time automation, smart task handling, and seamless user interaction. We have integrated speech recognition and natural language processing with modular execution pipelines within a PyQt5 interface, enabling the system to provide a swift and intuitive mechanism for interacting with desktop environments. Although the minimal and easy architecture is optimized for voice-only input, it provides low latency with reduced system complexity and smooth multimodal response delivery through text, audio, and image outputs. Our experimental usage shows that the assistant interprets the user queries more reliably and performs the automation tasks, retrieves information, and provides natural feedback by means of speech synthesis, and it's user-friendly and easy to manage for different kinds of tasks.

In general, project JARVIS serves as a practical, scalable implementation of real-time intelligent assistance that bridges the gap between traditional automation tools and conversational AI interfaces.

Although our current implementation has achieved robust real-time interaction and automation, several enhancements are required which can greatly extend the system capabilities. In the future we may focus on integrating more advanced large language models that can locally reduce dependency on cloud services and improve privacy of the users. We will incorporate contextual memory that would allow the assistant to maintain short-term conversational state and continuity with the user. Soon we will expand multimodal inputs such as gesture recognition or on-screen

object detection that would further improve user interaction. Also, we will integrate IoT device control that would extend JARVIS beyond the desktop to support smart home environments.

We can focus on improved personalization, multilingual speech models capable of understanding almost all languages, reinforcement-learning-based adaptation, and on-device wake-word detection that can promise directions toward the creation of a more autonomous, intelligent, and user-centric assistant. We will expand the security framework to include user authentication and the secure execution of tasks necessary for real-world deployment.

Those would significantly enhance the robustness, autonomy, and practical applicability of the JARVIS system for various operational scenarios in the future.

VI. ACKNOWLEDGEMENT

First and foremost, we would like to take this opportunity to thank our faculty mentor and teachers, the department of Computer Science and Engineering at KIET Group of Institutions, for their continuous guidance, technical support, and valuable insights during the development of this project. Resources, laboratory facilities, and most importantly the encouragement by the institution have been highly instrumental in successfully building the AI-Powered Desktop Assistant, also called JARVIS. We also extend our appreciation to peers and collaborators whose feedback helped us a lot in refining and improving the system overall.

VII. REFERENCES

- [1] R. Joshi, S. Ghimire, B. Paudel, and S. Chaudhary, "Personal AI Desktop Assistant," *International Journal of Information Technology Research and Applications (IJITRA)*, vol. 3, no. 2, pp. 45–52, 2023.
- [2] A. Jain, S. Gupta and R. Singh, "Voice Recognition System for Desktop Assistant," in *Proceedings of the 4th International Conference on Communication, Business and Visual Arts (ICCBV)*, 2022, pp. 112–118.
- [3] P. Sharma, N. Chauhan and S. Verma, "Design and Development of an AI-Based Personal Desktop Assistant," *International Journal of Computer Applications*, vol. 184, no. 25, pp. 10–16, 2022.
- [4] K. Arora, M. Singh and R. Patel, "A Survey on Speech-to-Text and Text-to-Speech Systems using Machine Learning," *International Journal of Speech Technology*, vol. 25, no. 3, pp. 765–778, 2023.
- [5] D. Kaur and P. Yadav, "Deep Learning Approaches for Virtual Assistant Systems: A Review," *Journal of Intelligent Systems*, vol. 32, no. 1, pp. 587–600, 2023.
- [6] S. Srivastava and A. Kalra, "Natural Language Processing Techniques for Human-Computer Interaction," in *Proceedings of the 2021 International Conference on Computational Intelligence and Knowledge Economy (ICCIKE)*, pp. 331–336, 2021.
- [7] H. Zhang, Y. Chen and L. Wu, "Automation Frameworks for Intelligent Desktop Agents," *International Journal of Automation and Smart Technology*, vol. 12, no. 4, pp. 254–262, 2022.

PROOF OF ACCEPTANCE



IEEE IC2PCT 2026: Paper Acceptance Notification and Registration Form | Paper ID- 805 | 2026 IEEE 6th International Conference on Computing, Power, and Communication Technologies | (MODE-OFFLINE) | Slot -1

1 message

Microsoft CMT <noreply@msr-cmt.org>
To: Abha Gangwar <abha.2226cse1148@kiet.edu>
Cc: ic2pct@galgotiasuniversity.edu.in

Thu, 5 Feb 2026 at 5:05 pm

Dear Prof./Dr./Mr./Ms. Abha Gangwar,

Congratulations!

We are pleased to inform you that your manuscript id-805 & titled "JARVIS:AI Powered Desktop Assistant Intelligent Automation and Real-Time Human-Computer Interaction" has been peer-reviewed and accepted for paper presentation, in OFFLINE Mode, at "2026 IEEE 6th International Conference on Computing, Power, and Communication Technologies", which will be held at Galgotias University, Greater Noida, India (203201), from May 15-17, 2025.

The conference is technically and financially & Technically sponsored by the IEEE Uttar Pradesh Section and the technically sponsored by IEEE Education Society.

Step to Follow:

1. Revise your manuscript as suggested by reviewers. You can check it in your cmt login.
2. Verify plagiarism levels in your final camera-ready paper after revision-ensure it stays below the specified limit.
3. Make payment of the Paper Publication Fee through NEFT/Internet Banking, applicable amount for Offline Author:<https://www.ic2pct.co.in/registration>
2. Fill the Registration form with all details and upload the final camera-ready paper using this Registration Form Link: <https://forms.gle/KPwX3vPoiqyCaTgI7>
3. Deadline for registration (Slot-wise): February 20, 2026, as it is based on a slot-wise decision. (No extension)(Any fees paid after the specified deadline will neither qualify for presentation nor be eligible for a refund.)

Important :

1. All the accepted and presented papers meeting IEEE Xplore's scope and quality standards will be submitted for inclusion in IEEE Xplore, ensuring global visibility and recognition for the authors.
2. Registration requires completing the shared Registration Form after payment. Payment alone does not constitute registration.
3. Any payment made without completing the registration form will be deemed incomplete and ineligible for presentation or refund.

Kindly log in to your CMT account to review the comments provided by the reviewers.

"It is essential that you incorporate all suggested revisions given by reviewers in your camera-ready manuscript to ensure compliance with the review process."
Failure to address the reviewers' comments may result in rejection. Additionally, please adhere to the plagiarism guidelines outlined below:

Important Guidelines:

1. Ensure that the similarity index (plagiarism) does not exceed 10% including references, with AI-generated content contributing no more than 15%, to qualify for submission to the IEEE Xplore Digital Library.
2. Complete the registration process by February 20, 2026, as it is based on a slot-wise decision.
3. Regularly check the conference website for updates on registration, camera-ready manuscript submission, copyright submission, PDF Express verification, and other important details.
4. Aim for a manuscript length of six pages. If Additional pages, pay the extra amount applicable, upto 2 pages.

Mandatory Requirements:

Registration through Google Form and presentation (offline) is compulsory, as per the conference guidelines.

Note: "If you missed registering during your designated slot, your paper registration will be placed on hold and considered later, subject to availability in the final limited registration count."

Should you have any queries, please do not hesitate to contact us.

With warm regards,
Organizing Committee, IC2PCT-2026
Galgotias University, Greater Noida, India

PLAGIARISM REPORT

PCSE03_Student

-  Quick Submit
-  Quick Submit
-  KIET Group of Institutions, Ghaziabad

Document Details

Submission ID
trnoid::1:3457180104

Submission Date
Jan 14, 2026, 9:28 AM GMT+5:30

Download Date
Jan 14, 2026, 9:32 AM GMT+5:30

File Name
Updated_Research_Paper_-_PCSE-03.pdf

File Size
1.7 MB

7 Pages

3,258 Words

19,432 Characters



Page 1 of 9 • Cover Page

Submission ID trnoid::1:3457180104



Page 2 of 9 • AI Writing Overview

Submission ID trnoid::1:3457180104

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.